

TLP:CLEAR

OPEN-SOURCE INTELLIGENCE

TECHNICAL REPORT

TeamPCP Supply Chain Attack Campaign

Multi-Ecosystem Credential-Chaining Operation

CVE-2026-33634 | CanisterWorm | Trivy | Checkmarx | LiteLLM | Telnyx

Report Date:	28 March 2026
Version:	2.0 (Add ATT&CK + Supply Chain Attack Lineage)
Classification:	TLP:CLEAR — Unlimited Distribution
Status:	ACTIVE — Campaign Ongoing
CVE Reference:	CVE-2026-33634 (CVSS 9.4)
CISA KEY:	Added 26 March 2026 — Due Date: 9 April 2026
Threat Actor:	TeamPCP (DeadCatx3 / PCPcat / PersyPCP / ShellForce)
Prepared By:	Douglas Mun with AI assistance

[Disclaimer](#)

[1. Executive Summary](#)

[2. Background](#)

[2.1 The Open-Source Supply Chain as Attack Surface](#)

[2.2 Threat Actor: TeamPCP](#)

[2.3 Precursor Events and Earlier CVE](#)

[3. Comprehensive Timeline](#)

[4. Technical Details of the Campaign](#)

[4.1 Campaign Mechanics: Credential Chaining](#)

[4.2 Initial Access: hackerbot-claw AI Agent](#)

[4.3 Trivy / GitHub Actions Malware \(Phase 2\)](#)

[4.3.1 GitHub Actions Tag Poisoning](#)

[4.3.2 The TeamPCP Cloud Stealer \(GitHub Actions Payload\)](#)

[4.3.3 Malicious Trivy Binary \(v0.69.4\)](#)

[4.4 CanisterWorm: Self-Propagating npm Worm \(Phase 3\)](#)

[4.4.1 Stage 1: The Postinstall Loader \(index.js\)](#)

[4.4.2 Stage 2: The Python Backdoor \(service.py\)](#)

[4.4.3 Stage 3: The Worm \(deploy.js\)](#)

[4.4.4 Worm Evolution: Self-Propagating Mutation](#)

[4.5 Kubernetes Wiper — Geopolitically Targeted Destruction \(Phase 3.5\)](#)

[4.5.1 Four-Path Decision Tree](#)

[4.5.2 Lateral Movement: SSH and Docker API Spread](#)

[4.6 LiteLLM PyPI Variant \(Phase 5\)](#)

[4.6.1 Three-Stage Malware Design](#)

[4.6.2 Accidental Discovery via Fork Bomb](#)

[4.6.3 Bot-Driven Issue Suppression](#)

[4.7 Checkmarx and OpenVSX Variant \(Phase 4\)](#)

[4.8 Telnyx PyPI Variant \(Phase 6\)](#)

[4.8.1 WAV Steganography Delivery](#)

[4.8.2 Platform-Specific Behaviour](#)

[4.8.3 Attribution and Credential Chain](#)

[4.9 Malware Evolution Summary](#)

[4.10 CVEs, Advisories, and Formal Tracking](#)

[4.11 Evidentiary Gaps: What Is Proven vs Inferred](#)

[5. MITRE ATT&CK Mapping](#)

[6. Indicators of Compromise \(IOCs\)](#)

[6.1 Network, C2, and Exfiltration Infrastructure](#)

[6.2 File, Service, and Repository Artefacts](#)

[6.3 Affected Software Versions and Exposure Windows](#)

[6.4 Hashes and Digests](#)

[6.5 Kubernetes, Container, and Runtime Behavioural Indicators](#)

[7. Current and Potential Impact](#)

[7.1 Impact on the Open-Source Ecosystem](#)

[7.2 Impact on Downstream Users and Commercial Software](#)

[7.3 Cache Persistence Risk](#)

[8. Impact Scoping: Comparison with SolarWinds and Log4j](#)

[8.1 Compared with SolarWinds](#)

[8.2 Compared with Log4j](#)

[8.3 Why TeamPCP Is More Dangerous Than a Normal “Bad Package” Incident](#)

[8.4 Systemic Architectural Failures That Enabled the Campaign](#)

[8.5 Highest-Risk Victims](#)

[8.6 Trivy Is Not Confined to Dev/UAT Environments](#)

[8.7 Direct vs Indirect Production Impact](#)

[8.8 Bottom Line: Impact Scoping](#)

[9. Supply Chain Attack Lineage: TeamPCP in Context](#)

[9.1 Evolution Comparison](#)

[9.2 The Credential Lineage Chain](#)

[9.3 Technique Evolution: What Each Campaign Learned](#)

[9.3.1 tj-actions/SpotBugs \(March 2025\): The Proof of Concept](#)

[9.3.2 s1ngularity / Nx \(August 2025\): AI-Assisted Reconnaissance](#)

[9.3.3 Shai-Hulud 1.0 and 2.0 \(September–November 2025\): The First npm Worm](#)

[9.3.4 TeamPCP \(February–March 2026\): The Convergence](#)

[9.4 The Acceleration Pattern](#)

[10. Threat Actor Profile and Assessed Motives](#)

[10.1 Assessed Motives](#)

[11. Recommended Course of Actions](#)

[11.1 Immediate Actions \(0–48 Hours\)](#)

[11.2 Short-Term Actions \(1–2 Weeks\)](#)

[11.3 Medium-Term Actions \(1–3 Months\)](#)

[12. Summary](#)

[13. References](#)

Disclaimer

This report is classified TLP:CLEAR and is approved for unlimited distribution. It is based entirely on open-source intelligence (OSINT) gathered from publicly available vendor advisories, government advisories (CISA KEV), security research publications, and community reporting. No classified, proprietary, or non-public information has been used in its preparation.

The campaign is still evolving. Some technical details, victim counts, and threat actor relationships may change as new information emerges. Where primary sources do not confirm attribution or follow-on actor relationships, they are labelled as assessed or unverified rather than fact.

This document does not constitute legal advice, nor does it represent the official position of any government agency. Organisations should conduct their own risk assessments and engage appropriate legal and technical counsel before taking remedial action. References to specific vendors, products, or services are for informational purposes and do not imply endorsement or criticism.

1. Executive Summary

The March 2026 TeamPCP campaign is best understood as a credential-driven software supply chain compromise, rather than a conventional single-CVE exploit. The attacker first retained or regained privileged access to trusted release automation, then poisoned real software distribution channels: Trivy releases and GitHub Actions, Docker Hub images, Checkmarx GitHub Actions and OpenVSX plugins, npm packages carrying the CanisterWorm, PyPI packages including LiteLLM, and, later, Telnyx. In each phase, the attacker reused stolen secrets to pivot into the next ecosystem, turning trusted CI/CD and package-publishing paths into malware-delivery and credential-theft infrastructure.

The most important technical lesson is that this campaign attacked trust anchors: mutable GitHub Action tags, release bots, package publisher credentials, container registries, and developer install paths. The malware family harvested environment variables, runner memory, SSH keys, cloud credentials, Kubernetes tokens, Docker auth, Terraform state, and other sensitive artefacts; encrypted them with AES-256-CBC and RSA-4096; exfiltrated them to the attacker's infrastructure; and, in some variants, established persistence via systemd or attempted Kubernetes-wide propagation.

For organisations globally, the key risk is not just the direct use of a compromised package. It is possible that software vendors, system integrators, AI teams, managed service providers, or government contractors executed poisoned tooling in build environments and then propagated signed or otherwise trusted downstream artefacts into production. CISA added CVE-2026-33634 to the Known Exploited Vulnerabilities catalogue on 26 March 2026, with a federal remediation due date of 9 April 2026. Section 8 of this report provides a detailed impact-scoping comparison with SolarWinds and Log4j to frame the severity and nature of this campaign accurately.

CRITICAL ASSESSMENT

Campaign Status: ACTIVE and ONGOING as of 28 March 2026.

Estimated Impact: Over 1,000 SaaS environments confirmed affected (Mandiant CTO, 25 March). Projections of 5,000–10,000 from secondary reporting.

Collaboration: Secondary reporting speculates that TeamPCP is linked to LAPSUS\$ and a ransomware group called Vect. No primary-source vendor or government confirmation of these partnerships has been identified. This should be treated as UNVERIFIED INTELLIGENCE, not a firm analytic conclusion.

Novel Techniques: First documented use of ICP blockchain canister as C2; first self-propagating npm worm in the wild; WAV steganography for payload delivery; AI-powered autonomous initial access via hackerbot-claw.

Formal Identifiers: CVE-2026-33634 (CVSS 9.4), CISA KEV, GHSA-69fq-xp46-6x23.

2. Background

2.1 The Open-Source Supply Chain as Attack Surface

Modern software development is fundamentally dependent on open-source software (OSS). Security scanning tools like Trivy, development libraries like LiteLLM, and communications SDKs like Telnyx are deeply embedded in enterprise CI/CD pipelines and production environments. These tools run with elevated privileges in build environments, have access to deployment credentials, and often execute automatically without human review. This trust relationship creates a high-value target for adversaries.

Docker's write-up on this incident is especially clear about a critical distinction: this was not fundamentally a dependency-graph problem but a credential-and-provenance problem, in which valid vendor credentials were used to push malicious content through legitimate channels. The usual defensive assumption that "official namespace equals safe" was undermined because the attacker compromised real projects and distribution channels, not just typo-squatted packages.

2.2 Threat Actor: TeamPCP

TeamPCP (also tracked as DeadCatx3, PCPcat, PersyPCP, ShellForce, and CipherForce) is a cloud-native cybercriminal operation that has been active since at least December 2025. Multiple researchers associate it with consistent infrastructure across phases: the same RSA key pair, tpcp.tar.gz naming convention, and tpcp-docs-prefixed GitHub repositories used as dead-drop staging. The actor maintains Telegram channels (@Persy_PCP and @teampcp) and embeds the string "TeamPCP Cloud stealer" in payloads. Snyk, Wiz, Datadog, Endor Labs, and multiple other security research organisations have tracked the full campaign.

The motive profile is assessed as primarily credential theft, access expansion, and monetisable downstream compromise. The malware prioritises cloud credentials, CI/CD secrets, registry tokens, and Kubernetes access, which are valuable for follow-on intrusion, data theft, republishing, extortion, resale, and infrastructure abuse. Some phases also exhibit performative or signalling behaviour: public taunting of victims, suppression of issues using botnet accounts, and visible campaign branding.

ANALYTIC NOTE: LAPSUS\$ AND VECT COLLABORATION CLAIMS

Secondary reporting (SecurityWeek, The Hacker News, MrCloudBook) speculates on TeamPCP's collaboration with LAPSUS\$ for extortion and an emerging ransomware group called Vect. However, no primary-source vendor advisory, government statement, or first-party security research firm has confirmed an operational partnership. These claims should be treated as UNVERIFIED INTELLIGENCE and monitored for corroboration, not cited as fact in operational assessments.

2.3 Precursor Events and Earlier CVE

The campaign follows an escalating pattern of CI/CD-focused supply chain attacks: SolarWinds (2020), Codecov (2021), ua-parser-js (2021), and tj-actions/changed-files (2025), which pioneered the tag-poisoning technique later used against Trivy.

Background context also includes CVE-2026-26189 (GHSA-9p44-j4g5-cfx5), a command-injection vulnerability in trivy-action discovered before the March campaign. While that is a separate issue, it illustrates that the Trivy GitHub Actions surface had prior known

weaknesses. The March 19 wave, however, was primarily a retained-credential and release-poisoning event, not exploitation of that specific CVE in isolation.

3. Comprehensive Timeline

The following timeline consolidates events from primary vendor advisories (Aqua Security, Checkmarx, LiteLLM, Telnix, Docker, CISA) and validated security research (Wiz, Datadog, Sysdig, Snyk, Aikido, Microsoft, Endor Labs, JFrog, Socket, Palo Alto Networks, Arctic Wolf, SANS, Kaspersky, ARMO, StepSecurity, Semgrep, Trend Micro, SafeDep). Precise exposure windows are drawn from vendor-published data.

Date / Window	Event
Late 2025	TeamPCP begins operations, scanning for exposed Docker APIs, Kubernetes control planes, Ray dashboards, and Redis services across the UAE, Canada, South Korea, Serbia, the US, and Vietnam. Cyble profiles the group as a cloud-native exploitation platform.
20 Feb 2026	The hackerbot-claw GitHub account has been created. Its profile describes a vulnerability pattern index covering 9 classes and 47 sub-patterns. StepSecurity later documents this as an AI-powered autonomous attack tool built on OpenCLAW that systematically scanned repositories for exploitable GitHub Actions workflows.
27–28 Feb 2026	Phase 1 — Initial Access. hackerbot-claw exploits a misconfigured pull_request_target workflow (apidiff.yaml, present since October 2025 and flagged by Boost Security's poutine tool on 29 November 2025) in Aqua Security's Trivy repository. The bot exfiltrates a PAT (ORG_REPO_TOKEN) with write access to all 33+ repositories in the Aqua Security GitHub organisation. The attacker validates access by creating branches with emoji patterns, privatises the repo, deletes 178 releases, and publishes a malicious VS Code extension.
1 Mar 2026	Aqua Security discloses the first incident and begins credential rotation. Aqua later acknowledges the rotation was not atomic (not all credentials were revoked simultaneously), leaving a window in which refreshed credentials may also have been exposed. This is the root cause of the March 19 attack.
19 Mar 2026 ~17:43 UTC	Phase 2 — Supply Chain Injection. TeamPCP uses retained credentials to force-push 76 of 77 version tags in aquasecurity/trivy-action and all 7 tags in aquasecurity/setup-trivy to malicious commits. The compromised aqua-bot account triggers publication of malicious Trivy v0.69.4 to GitHub Releases, Docker Hub, GHCR, Amazon ECR, deb/rpm, and get.trivy.dev. The legitimate Trivy scan runs in parallel, masking the compromise. Aqua reports the core incident was identified and contained by ~20:38 UTC.
Trivy Exposure Windows	trivy binary v0.69.4: ~18:22 UTC to ~21:42 UTC on 19 March. trivy-action: ~17:43 UTC on 19 March to ~05:40 UTC on 20 March. setup-trivy: ~17:43 UTC to ~21:44 UTC on 19 March. (Per Aqua Security advisory.)
19–20 Mar 2026	44 Aqua Security GitHub repositories are renamed in a 79-second scripted burst with the tcp-docs- prefix and the description "TeamPCP Owns Aqua Security." Imposter commits spoof maintainer identities with forged author metadata.
20 Mar 2026 20:45 UTC	Phase 3 — CanisterWorm deployed on npm. Aikido Security detects a large-scale compromise: dozens of packages across multiple organisations receiving unauthorised patch updates with identical malicious code. The worm uses ICP blockchain canisters for C2. 28 packages are compromised within 60 seconds within the @EmilGroup scope. Affected npm namespaces include @EmilGroup, @opengov, @teale.io, @airtm, and @pypestream.
22 Mar 2026	Malicious Trivy Docker Hub images v0.69.5 and v0.69.6 were published without corresponding GitHub releases. Docker confirms that the affected pulls include v0.69.4, v0.69.5, v0.69.6, and the latest during the relevant window. Separately, researchers

Date / Window	Event
	observe TeamPCP deploying WAV steganography to deliver payloads in a Kubernetes wiper variant targeting Iranian systems.
23 Mar 2026	Phase 4 — Checkmarx compromised. Checkmarx discloses a compromise of ast-github-action and kics-github-action, as well as two malicious OpenVSX plugins (ast-results 2.53.0 and cx-dev-assist 1.7.0). Exposure windows: OpenVSX 02:53–15:41 UTC; KICS GitHub Action 12:58–16:50 UTC. Remediation was completed the same day. New C2 domain checkmarx[.]zone activated. This proves the campaign has crossed from OSS-only projects into commercial AppSec tooling.
24 Mar 2026 10:39–10:52 UTC	Phase 5 — LiteLLM compromised on PyPI. Versions 1.82.7 and 1.82.8 were published using credentials stolen from LiteLLM's CI pipeline (which ran unpinned Trivy). v1.82.8 adds a .pth file that executes on any Python invocation, even without importing LiteLLM. The official LiteLLM Proxy Docker image was NOT affected (it pinned dependencies). Attackers use 73 compromised developer accounts to post 88 bot comments in 102 seconds, suppressing community reporting. PyPI quarantines packages within approximately 3–5 hours.
25 Mar 2026	Mandiant CTO Charles Carmakal states that over 1,000 SaaS environments are actively dealing with this threat actor. CanisterWorm count expands to 135+ malicious package artefacts across 64+ unique npm packages.
26 Mar 2026	CISA adds CVE-2026-33634 to the Known Exploited Vulnerabilities (KEV) catalogue, with a federal remediation due date of 9 April 2026.
27 Mar 2026 03:51–04:07 UTC	Phase 6 — Telnyx compromised on PyPI. Versions 4.87.1 and 4.87.2 were published with malicious code hidden inside WAV audio files (steganography). C2 at 83[.]142[.]209[.]203:8080. Telnyx confirms the compromise was limited to the PyPI distribution channel; their production platform and APIs were not affected. PyPI quarantines the packages by 10:13 UTC.
28 Mar 2026	Campaign remains active and ongoing. Endor Labs and others assess additional targets expected to be available across PyPI, RubyGems, crates.io, and Maven Central. TeamPCP has standardised WAV steganography as a delivery technique.

4. Technical Details of the Campaign

This section provides a detailed technical analysis of each phase of the campaign, drawing primarily on research from Aikido Security, Wiz, Datadog Security Labs, Sysdig, Semgrep, Trend Micro, Microsoft, Snyk, Endor Labs, JFrog, Socket, and SafeDep. The level of detail is intended to support SOC hunting, incident response triage, and architectural review.

4.1 Campaign Mechanics: Credential Chaining

The core pattern was: compromise a trusted automation or publisher credential, push malicious artefacts through an official channel, steal more credentials from downstream users, then reuse those newly stolen secrets to compromise the next project. Each phase yielded credentials that unlocked the next target. This is not a single incident; it is a self-reinforcing credential-powered chain that crosses ecosystem boundaries. Datadog, Sysdig, Aqua, Docker, and LiteLLM all describe a pivot pattern built on stolen CI/CD pipelines and the publication of secrets, rather than a single isolated software defect.

4.2 Initial Access: hackerbot-claw AI Agent

The campaign's initial access vector was an AI-powered autonomous attack tool operating under the GitHub account `hackerbot-claw` (created 20 February 2026). Its profile self-describes as “an autonomous security research agent powered by claude-opus-4-5,” claiming to support “9 classes and 47 sub-patterns” for vulnerability scanning. Whether the AI attribution is genuine or performative, the operational pattern — systematic scanning of thousands of repos for exploitable workflow configurations — is clearly automated at scale. StepSecurity documented `hackerbot-claw` as achieving code execution across multiple OSS repositories (including Microsoft, Datadog, and CNCF projects) and exfiltrating `GITHUB_TOKEN` values to `hackmoltrepeat[.]com/moult` and `recv.hackmoltrepeat[.]com`.

The bot specifically targeted `pull_request_target` workflow misconfigurations. When a GitHub Actions workflow uses `pull_request_target`, it runs with the base repository's secrets and permissions, but can be tricked into checking out and executing code from a fork's pull request. Trivy's repository had a vulnerable workflow (`apidiff.yaml`) since October 2025; Boost Security's poutine tool flagged it on 29 November 2025 — over three months before it was exploited. The bot opened PR #10254 with a legitimate-sounding branch name, modified the GitHub Actions setup, and extracted the `ORG_REPO_TOKEN` PAT, which had repo-scope (and likely admin-scope) access across all 33+ Aqua Security repositories. The token was used in at least 33 workflows. This was not a human at a keyboard; it was an automated system that could read YAML files, identify misconfigurations, craft convincing pull requests, and exfiltrate secrets autonomously.

4.3 Trivy / GitHub Actions Malware (Phase 2)

On 19 March 2026, beginning at approximately 17:43 UTC, TeamPCP used the retained credentials to execute a multi-pronged attack. The attacker made imposter commits spoofing maintainer `DmitriyLewen` (on `aquasecurity/trivy`) and `Guillermo Rauch / rauchg` (on `actions/checkout`), with forged author names, emails, timestamps, and commit messages. This is possible because Git's commit identity is self-declared — there is no cryptographic verification of author metadata by default.

4.3.1 GitHub Actions Tag Poisoning

The attacker force-pushed 76 of 77 version tags in aquasecurity/trivy-action (every tag from v0.0.1 through v0.34.2; only v0.35.0 was protected by GitHub's immutable releases) and all 7 tags in aquasecurity/setup-trivy to point to malicious commits. Any CI/CD pipeline that referenced these actions by version tag automatically began running attacker-controlled code on its next execution, with no visible changes to release metadata or workflow files. Old tags without a "v" prefix (0.0.1–0.34.2) should be considered compromised; new v-prefixed tags (v0.0.1–v0.34.2) created post-remediation point to original legitimate commits.

4.3.2 The TeamPCP Cloud Stealer (GitHub Actions Payload)

Wiz's analysis describes the poisoned trivy-action and setup-trivy payload as a three-stage credential stealer that runs before the legitimate Trivy scan, so pipeline output looks completely normal:

- **Stage 1 — Runner Process Memory Scraping:** The payload identifies the GitHub Actions runner process by executing `pgrep -f Runner.Worker` and `pgrep -f Runner.Listener`. It then reads `/proc/<pid>/mem` to dump the Runner. Worker process memory, extracting secrets that match GitHub runner secret patterns. This technique captures secrets that may never appear in environment variables, filesystem paths, or configuration files — they exist only in memory during workflow execution. Microsoft's Defender hunt guidance provides specific `CloudProcessEvents` and `DeviceProcessEvents` queries targeting these process patterns.
- **Stage 2 — Filesystem Credential Sweep (50+ Paths):** The stealer searches a comprehensive list of credential locations across multiple cloud providers and development tools. AWS: `~/.aws/credentials`, environment variables `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`, EC2 IMDS tokens via metadata service. GCP: service account JSON files, `GOOGLE_APPLICATION_CREDENTIALS` environment variable (the payload specifically runs `cat $GOOGLE_APPLICATION_CREDENTIALS 2>/dev/null`). Azure: service principal credentials. SSH: all private keys in `~/.ssh/`. Git: credentials from `~/.gitconfig` and credential helpers. GitHub: PATs from environment variables and CI/CD secrets. Kubernetes: kubeconfig files from `~/.kube/config`, service account tokens, cluster API keys. Docker: `~/.docker/config.json` (Docker explicitly warns this file is a high-value target). npm: `~/.npmrc` and `NPM_TOKEN` environment variables. PyPI: `~/.pypirc` and `TWINE_PASSWORD`. Additionally: Terraform state files, `.env` files, shell history, and cryptocurrency wallets.
- **Stage 3 — Encrypted Exfiltration with Fallback:** All harvested data is encrypted with AES-256-CBC using a random 32-byte session key. The session key is then encrypted with RSA-4096/OAEP using the attacker's embedded public key. Both are bundled into `tpcp.tar.gz` and exfiltrated via HTTP POST to `scan.aquasecurtiy[.]org` (a typosquatted domain using "securtiy" instead of "security") with the header `X-Filename: tpcp.tar.gz`. If the primary exfiltration fails, the malware executes a fallback mechanism: it uses the victim's `GITHUB_TOKEN` to create a public repository named `tpcp-docs` in the victim's GitHub organisation and uploads the encrypted stolen data there. The presence of a `tpcp-docs` repository in any organisation is therefore a confirmed indicator of successful credential exfiltration.

4.3.3 Malicious Trivy Binary (v0.69.4)

In parallel with the GitHub Actions tag poisoning, the attacker triggered Trivy's standard release machinery to publish a backdoored v0.69.4 binary to GitHub Releases, Docker Hub, GHCR, Amazon ECR, deb and rpm packages, and get.trivy.dev. The malicious binary launched legitimate Trivy scanning functionality alongside the malicious process, ensuring that developers and CI/CD pipelines saw normal scan results. At the same time, credentials were being exfiltrated in the background.

On non-GitHub-Actions environments (developer workstations, cluster-side scanners, container hosts), the binary attempted persistence by writing a Python dropper (sysmon.py) to `~/.config/sysmon/` and installing a systemd user service (sysmon.service) that polled the same ICP canister (tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io) used by CanisterWorm for follow-on payloads. This shared C2 infrastructure is a key attribution indicator linking the Trivy binary compromise to the subsequent npm and PyPI phases. Additional malicious Docker images, v0.69.5 and v0.69.6, were pushed to Docker Hub on 22 March without corresponding GitHub releases, exploiting the disconnect between source repositories and artefact registries.

DOCKER HUB: CACHE PERSISTENCE WARNING

Docker explicitly warns that even after malicious upstream content is removed, registry mirrors, artefact proxies (including Nexus and Artifactory), and local caches may still hold the compromised bytes. Docker advises searching local stores, registry mirrors, and artefact proxy caches for compromised Trivy digests. Docker also notes that mounting the Docker socket in a compromised Trivy container can lead to a root-equivalent compromise on the host.

4.4 CanisterWorm: Self-Propagating npm Worm (Phase 3)

CanisterWorm is the first publicly documented self-propagating worm in the npm ecosystem. Aikido Security researcher Charlie Eriksen first detected it on 20 March 2026 at 20:45 UTC, when dozens of packages across multiple organisations began receiving unauthorised patch updates containing identical malicious code. The worm uses a three-stage architecture: a Node.js postinstall loader, a persistent Python backdoor, and an ICP-hosted dead-drop C2 with remote payload rotation.

4.4.1 Stage 1: The Postinstall Loader (index.js)

The entry point is a malicious postinstall script in package.json that activates automatically during npm install. The entire execution is wrapped in a try/catch block, ensuring npm install succeeds normally on all platforms regardless of whether the malicious logic fires. The loader only activates on Linux systems with systemd. It performs four actions: (1) decodes an embedded base64-encoded Python backdoor; (2) writes it to `~/.local/share/pgmon/service.py`; (3) creates a systemd user service unit file at `~/.config/systemd/user/pgmon.service` with `Restart=always`, `RestartSec=5`, and `After=default.target`; and (4) runs `systemctl --user daemon-reload`, then enables and starts the service. No root privileges are required — the entire persistence chain operates within the user's systemd scope. All artefacts use PostgreSQL-themed naming (pgmon, pglog, .pg_state) to blend in on developer machines.

4.4.2 Stage 2: The Python Backdoor (service.py)

The persistent Python backdoor uses only standard library modules (`urllib.request`, `os`, `subprocess`, `time`), so nothing needs to be installed. Its behaviour is designed for stealth and sandbox evasion:

- **Initial sleep:** Waits 300 seconds (5 minutes) before any network activity, long enough to outlast most sandbox environments.
- **C2 polling:** Function `g()` contacts the ICP canister at `tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io` with a spoofed Mozilla/5.0 User-Agent. The canister does not serve malware directly; it returns a URL as plain text pointing to wherever the real binary is currently hosted. Polling occurs every ~50 minutes (3,000 seconds).
- **Payload delivery:** Function `e()` fetches the URL returned by the canister, saves the binary to `/tmp/pglog`, marks it executable (`chmod 0o755`), and launches it in a fully detached process (`start_new_session=True`). The URL is saved to `/tmp/.pg_state` to prevent re-downloading the same payload.
- **Kill switch:** If the URL contains `youtube.com`, the script skips execution. This is the canister's dormant state. The attacker arms the fleet by pointing the canister to a real binary, and disarms it by switching back to a YouTube link. At the time of discovery, the canister was returning a Rickroll YouTube link — the entire infected fleet was dormant but fully operational.
- **Payload rotation:** The ICP canister exposes three methods: `get_latest_link` (retrieve the current payload URL), `http_request` (serve the URL to the backdoor), and `update_link` (allow the attacker to rotate to a new payload). The attacker can therefore change what executes on all infected machines at any time, without republishing any npm package and without triggering any detection in the npm registry.

4.4.3 Stage 3: The Worm (deploy.js)

The `deploy.js` script transforms CanisterWorm from a backdoor into a self-propagating worm. It is a standalone tool that the attacker runs with stolen tokens to maximise blast radius. Aikido notes it appears to be entirely AI-generated (“vibecoded”) with no obfuscation. The worm: (1) reads `NPM_TOKENS` (comma-separated) or `NPM_TOKEN` from environment variables, processing each independently; (2) calls the `npm /-/whoami` endpoint to resolve the associated username for each token; (3) uses the `npm search API (/-/v1/search` with `maintainer:<username>`, paginated in batches of 250) to enumerate every publishable package; (4) fetches the current latest version and increments the patch number; (5) fetches each target package's original README before publishing to preserve appearances; and (6) runs `npm publish --access public --tag latest` for each package with the malicious payload injected. In the @EmilGroup scope, 28 packages were compromised in under 60 seconds.

4.4.4 Worm Evolution: Self-Propagating Mutation

A subsequent mutation detected in `@teale.io/eslint-config` versions 1.8.11 and 1.8.12 incorporated the propagation functionality directly into the `postinstall` hook via a `findNpmTokens()` function. This eliminated the need for the attacker to run `deploy.js` with stolen tokens manually. Instead, the `postinstall` script itself installs the persistent backdoor. Immediately attempts to locate all npm tokens in the developer's environment and spawns the worm as a fully detached background process. This makes every compromised install an automatic and autonomous propagation vector. The `detached: true` and `.unref()` flags ensure the worm continues running even after npm install finishes.

4.5 Kubernetes Wiper — Geopolitically Targeted Destruction (Phase 3.5)

On 22 March 2026, Aikido Security discovered a new payload in the TeamPCP arsenal that uses the same ICP canister (tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io) documented in the CanisterWorm campaign. This variant adds geopolitically targeted destructive capability. The stager script downloads kubectl if not already available, then fetches kube.py from a Cloudflare tunnel domain (initially souls-entire-defined-routes.trycloudflare[.]com/kamikaze.sh, later split into separate stager and payload files).

4.5.1 Four-Path Decision Tree

The payload executes a simple but devastating decision tree based on two checks. First, it detects Kubernetes by checking for `/var/run/secrets/kubernetes.io/serviceaccount` or the `KUBERNETES_SERVICE_HOST` environment variable. Second, it identifies Iranian systems by reading `/etc/timezone` or querying `timedatectl` for Asia/Tehran or Iran timezone values, or checking the `LANG` environment variable for `fa_IR` locale:

Environment	Iran Detected	Action
Kubernetes	Yes	Deploy the destructive DaemonSet <code>host-provisioner-iran</code> across all nodes, including the control plane. Container named <code>kamikaze</code> mounts host root filesystem at <code>/mnt/host</code> , deletes all top-level directories (<code>find /mnt/host -maxdepth 1 -not -name mnt -exec rm -rf {} +</code>), then force-reboots every node via <code>chroot /mnt/host reboot -f</code> .
Kubernetes	No	Deploy persistent DaemonSet <code>host-provisioner-std</code> across all nodes. Installs the CanisterWorm backdoor as a system-level service on every node via <code>chroot</code> into the host filesystem.
Non-Kubernetes	Yes	Execute <code>rm -rf / --no-preserve-root</code> (complete host destruction).
Non-Kubernetes	No	Exit silently. Nothing happens.

Both DaemonSets use a security context with `hostNetwork: true`, `hostPID: true`, `privileged: true`, and an operator: `Exists` toleration to ensure scheduling on every node, including tainted control-plane nodes.

4.5.2 Lateral Movement: SSH and Docker API Spread

A later variant adds network-based lateral movement. It parses SSH authentication logs (`/var/log/auth.log` and `/var/log/secure`) to extract successful login pairs (usernames and IPs), steals SSH private keys from `~/.ssh/`, and uses them to spread to other machines on the local /24 subnet with `StrictHostKeyChecking=no` to suppress host key verification prompts. It also scans for exposed Docker APIs on port 2375 and creates privileged Alpine containers with the host root mounted at `/` to deliver the same payload. Both lateral movement methods carry the Iran timezone check — Iranian targets are wiped, while all others receive the persistent CanisterWorm backdoor. Cloudflare tunnel domains rotated across iterations, and the PostgreSQL-themed camouflage evolved: service names moved from `pgmon` to `internal-monitor` to `pgmonitor`, with install paths shifting accordingly.

4.6 LiteLLM PyPI Variant (Phase 5)

LiteLLM's official advisory confirms that versions 1.82.7 and 1.82.8 were compromised after credentials were stolen from the Trivy dependency used in LiteLLM's CI/CD scanning workflow, which pulled Trivy from apt without a pinned version. The compromised action exfiltrated the PYPI_PUBLISH token from the GitHub Actions runner environment. Importantly, the official LiteLLM Proxy Docker image was NOT affected because it pinned dependencies. LiteLLM is an open-source LLM proxy gateway with over 40,000 GitHub stars and approximately 95 million monthly downloads on PyPI (~3.6 million per day). Wiz Research found LiteLLM present in 36% of the cloud environments they monitor. A LiteLLM deployment typically stores API keys for every LLM provider the organisation uses (OpenAI, Anthropic, Azure, Google Vertex AI, AWS Bedrock, and others), making it a single point of compromise for the organisation's entire AI infrastructure.

4.6.1 Three-Stage Malware Design

Semgrep and Trend Micro describe the LiteLLM malware as more aggressive than the Trivy variant, with a three-stage design:

- **Orchestrator:** Injected into LiteLLM/proxy/proxy_server.py (v1.82.7) and additionally as a .pth file LiteLLM_init.pth (v1.82.8). The orchestrator decodes a base64-embedded payload and executes it. The .pth mechanism in v1.82.8 is particularly dangerous: Python's startup path automatically executes .pth files, meaning the malicious code runs on any Python invocation, even without importing LiteLLM. Simply having LiteLLM installed triggers execution.
- **Harvester:** Collects credentials from the same comprehensive list as the Trivy stealer (AWS, GCP, Azure, Kubernetes, SSH, Docker, Terraform, .env, shell history), encrypts with AES-256-CBC + RSA-4096 OAEP, and POSTs the tpcp.tar.gz bundle to models.LiteLLM[.]cloud.
- **Persistence and Kubernetes Lateral Movement:** If running in a Kubernetes environment with sufficient RBAC privileges, the malware creates privileged DaemonSets across all cluster nodes, mounts host filesystems via hostPath: /, and plants host-level backdoors. It also attempts to enumerate and dump all Kubernetes secrets cluster-wide (kubectl get secrets --all-namespaces -o json || true). This converts a single compromised pod into a full cluster compromise.

4.6.2 Accidental Discovery via Fork Bomb

The LiteLLM compromise was discovered accidentally, not by monitoring. Callum McMahon, a research scientist at FutureSearch, was testing a Cursor MCP plugin that pulled in LiteLLM as a transitive dependency. A bug in the malware's .pth file caused recursive subprocess spawning — the .pth file spawned a subprocess that itself triggered the .pth file, creating a fork bomb that crashed the developer's machine. Without this accidental detection, the persistence mechanism could have operated indefinitely. PyPI quarantined the package within approximately three hours, but in that window, anyone running pip install LiteLLM without version pinning would have pulled the weaponised version. LiteLLM averages approximately 3.6 million PyPI downloads per day.

4.6.3 Bot-Driven Issue Suppression

When community members began reporting the compromise in GitHub issue #24512, the attackers deployed a botnet, posting 88 comments from 73 unique accounts within a 102-second window (12:44–12:46 UTC). Snyk analysis found 76% account overlap with the botnet used during the earlier Trivy disclosure. Using the compromised maintainer account (krrishdholakia), the attackers closed issue #24512 as “not planned” and made commits to unrelated repositories with the message “teampcp update.” The community opened a parallel tracking issue (#24518) and continued discussion on Hacker News (324 points). The affected versions were on PyPI for approximately 3–5 hours before quarantine.

4.7 Checkmarx and OpenVSX Variant (Phase 4)

Checkmarx confirms the attacker force-pushed malicious tags to ast-github-action (before v2.3.33) and kics-github-action (before v5.0.20), and published two malicious OpenVSX plugins: ast-results-2.53.0.vsix and cx-dev-assist-1.7.0.vsix. The KICS payload used a setup.sh stealer tied to the C2 domain checkmarx[.]zone and fell back to creating a docs-tpcp repository with the victim’s GITHUB_TOKEN if direct C2 failed — the same fallback pattern from the Trivy phase. Sysdig confirmed that the payload, encryption scheme, and tpcp.tar.gz naming convention were identical to those in the Trivy phase.

This phase is strategically critical because it demonstrates the campaign shifted from OSS-only volunteer projects to commercial AppSec tooling and IDE/plugin distribution. Enterprise developers consuming “official” Checkmarx extensions or CI actions were directly exposed. Trend Micro additionally identified a checkmarx-util-1.0.4 artefact delivered directly from checkmarx[.]zone/raw rather than the public npm registry, indicating the attacker was also serving malicious packages from their own infrastructure alongside the hijacked official channels.

Datadog’s timeline analysis places the Checkmarx compromise as the bridge to the LiteLLM incident: by 23 March, the attacker had demonstrated a recurring pattern targeting GitHub Actions, package registries, and developer tooling across multiple vendor environments. The 13-minute gap between LiteLLM v1.82.7 (10:39 UTC) and v1.82.8 (10:52 UTC) on 24 March indicates the attacker was actively iterating during the attack window, adding the more aggressive .pth injection vector in the second release.

4.8 Telnyx PyPI Variant (Phase 6)

Telnyx confirms compromised versions 4.87.1 (03:51 UTC on 27 March) and 4.87.2 (04:07 UTC) were limited to the PyPI distribution channel; their production platform and APIs were not compromised. This phase introduces a new delivery technique: live payload fetching through WAV audio file steganography.

4.8.1 WAV Steganography Delivery

The malicious code injected into telnyx/_client.py (74 lines added to the legitimate module) triggers at the module scope — it executes on import of telnyx, with no user action required. A helper function _d() decodes a 4,436-character base64-encoded payload variable _p. The execution forks a detached child process (creates a new session leader, breaking the process away from the parent so it persists after the importing application exits). The detached script downloads ringtone.wav from the C2 (http://83[.]142[.]209[.]203:8080/ringtone.wav), which is a valid WAV audio file with the malicious payload embedded in the audio frame data. The choice of a .wav file format, suited to the Telnyx SDK’s telecommunications purpose, demonstrates the

attacker's awareness of the target ecosystem. The payload is extracted from the WAV data and piped into a fresh Python interpreter via `subprocess.run([sys.executable, "-"], input=payload)`.

4.8.2 Platform-Specific Behaviour

The Telnix payload exhibits a strategic behavioural split by operating system:

- **Windows:** Downloads `hangup.wav` from the same C2, extracts a native binary, drops it as `msbuild.exe` in `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\` (with an accompanying `.lock` file). This provides persistence across reboots via the Windows Startup folder, giving TeamPCP long-term repeatable access.
- **Linux/macOS:** Extracts and executes a credential harvester from `ringtone.wav` in-memory. The harvester collects credentials, encrypts them with the same AES-256-CBC + RSA-4096 OAEP scheme using the identical RSA public key from the LiteLLM compromise, and exfiltrates as `tpcp.tar.gz` via HTTP POST to `83[.]142[.]209[.]203:8080` with the header `X-Filename: tpcp.tar.gz`. Critically, there is no persistence mechanism on Linux/macOS: the temporary directory and all artefacts are recursively deleted after exfiltration. This is a deliberate stealth choice — extract credentials and leave no trace.

JFrog's analysis raises an additional concern: since the payload is fetched at runtime from a plain HTTP C2 (not HTTPS), any man-in-the-middle attacker could respond with their own `ringtone.wav` containing arbitrary payload. Unlike the XZ backdoor (which performed signature validation on downloaded payloads), the Telnix variant performs no validation. This means the malicious versions remain dangerous even if the original C2 goes offline.

4.8.3 Attribution and Credential Chain

Endor Labs and Socket assess that the most likely credential theft vector was the LiteLLM compromise itself: TeamPCP's harvester swept environment variables, `.env` files, and shell histories from every system that imported LiteLLM. If any developer or CI pipeline had both LiteLLM installed and access to the telnix PyPI token, that token was already in TeamPCP's hands. The three-day gap between LiteLLM (24 March) and Telnix (27 March) aligns with the time required to sift through stolen credentials and select the next target. Attribution confidence is HIGH based on: identical RSA-4096 public key, identical AES-256-CBC + RSA OAEP encryption scheme, `tpcp.tar.gz` archive naming and `X-Filename` header, and consistent registrar (Spaceship, Inc.) and hosting provider (DEMENIN B.V.) across campaign domains.

4.9 Malware Evolution Summary

The campaign demonstrates clear iterative development across phases. Each wave introduced new capabilities while retaining the core credential-harvesting and encrypted exfiltration infrastructure:

Phase	New Capabilities Added	Persistence
Trivy (19 Mar)	Runner memory scraping; 50+ path sweep; encrypted exfil; GitHub repo fallback; legitimate scan runs in parallel	systemd user service (sysmon.py)
CanisterWorm (20 Mar)	First npm self-propagating worm; ICP blockchain C2; automated npm publish across scopes; README preservation	systemd user service (pgmon)

Phase	New Capabilities Added	Persistence
K8s Wiper (22 Mar)	Geopolitical targeting (Iran); DaemonSet cluster-wide deployment; SSH lateral movement; Docker API exploitation; WAV steganography (first use)	DaemonSets + systemd (internal-monitor)
Checkmarx (23 Mar)	Cross-vendor pivot; commercial AppSec tooling; OpenVSX IDE plugin poisoning; brand impersonation C2 domain	Same stealer + fallback pattern
LiteLLM (24 Mar)	.pth auto-execution on any Python invocation; K8s lateral movement via DaemonSets; bot-driven issue suppression (73 accounts, 88 comments, 102 seconds)	systemd + .pth file + K8s DaemonSets
Telnix (27 Mar)	WAV steganography as a standard technique; live C2 payload fetching; no-persistence stealth on Linux; Windows Startup folder persistence; platform-specific split behaviour	Windows only (msbuild.exe); Linux ephemeral

4.10 CVEs, Advisories, and Formal Tracking

Identifier	Description	Status
CVE-2026-33634	Aquasecurity Trivy Embedded Malicious Code (CVSS 9.4, CWE-506)	CISA KEV — Due 9 Apr 2026
CVE-2026-26189 / GHSA-9p44-j4g5-cfx5	Trivy Action command injection (background context)	Prior issue
GHSA-69fq-xp46-6x23	Trivy ecosystem supply chain compromise	Critical
GHSA-cxm3-wv7p-598c	Trivy supply chain attack	Critical
PYSEC-2026-2	LiteLLM PyPI advisory	Critical
SNYK-PYTHON-LiteLLM-15762713	LiteLLM compromise (Snyk)	Critical
MSC-2026-3271	CanisterWorm (Mend.io tracking)	Critical
XRAY-957731	Telnix PyPI compromise (JFrog)	Critical

4.11 Evidentiary Gaps: What Is Proven vs Inferred

Intellectual honesty requires acknowledging where the public evidence supports firm conclusions and where it relies on inference. Two significant gaps exist in the current reporting, and this report should be transparent about both.

- Gap 1 — npm Token Provenance:** The widely repeated narrative that npm tokens exfiltrated from Trivy-compromised CI/CD pipelines seeded the CanisterWorm campaign is a high-confidence inference rather than a forensic fact. No security vendor has published evidence tracing a specific npm token from a specific compromised pipeline to a specific malicious npm publication. The evidence supporting the connection is circumstantial: shared threat-actor attribution (same ICP canister C2), a sequential timeline (~27 hours), and credential-stealer capability. However, CrowdStrike's detailed payload analysis does not list npm tokens among the enumerated credential types; Socket.dev explicitly declined to provide firm threat actor attribution; and none of the affected npm scope maintainers (@EmilGroup, @opengov, @teale.io) has publicly confirmed running Trivy in their CI/CD pipelines. Alternative token sources — including

prior Operation PCPcat compromises (December 2025), direct developer machine compromise, or CanisterWorm's own findNpmTokens() self-propagation from a single initial seed of unknown origin — have not been publicly explored.

- **Gap 2 — Behavioural Discontinuity (Two Actors or One?):** The March 1 behaviour (destructive, noisy — deleting 178 releases, vandalising the README with “Just researchmaxxed the PAT,” causing visible outages) is fundamentally inconsistent with the March 19 behaviour (sophisticated, patient — running the legitimate Trivy scan afterward, using vendor-specific typosquat domains, AES/RSA encryption, artifact cleanup). If the goal was silent credential harvesting, the last thing a rational attacker would do is destroy releases and draw investigation. Four explanations exist: (a) an access-broker model where hackerbot-claw/MegaGame10418 sold or transferred the credentials to TeamPCP; (b) the same actor with evolving objectives; (c) deliberate disruption as reconnaissance to observe Aqua's incident response; or (d) the non-atomic rotation changed the calculus when the attacker discovered the door stayed open. Boost Security Labs linked MegaGame10418 to hackerbot-claw via timeline overlap, but the connection from there to TeamPCP is described as “circumstantial.” The access-broker model is the most parsimonious explanation. This report does not resolve which explanation is correct, but acknowledges the discontinuity.

5. MITRE ATT&CK Mapping

The following mapping covers observed techniques across the full campaign lifecycle. It draws from CrowdStrike's payload analysis, Wiz's threat actor profiling, Sysdig's runtime detection research, Microsoft's Defender hunt guidance, and Aikido's CanisterWorm/wiper analysis. Technique IDs follow the Enterprise ATT&CK v15 matrix.

Tactic	Technique ID	Technique Name	Campaign Evidence
Reconnaissance	T1596	Search Open Technical Databases	hackerbot-claw systematically scanning thousands of repos for pull_request_target misconfigurations
Resource Development	T1583.001	Acquire Infrastructure: Domains	Vendor-specific typosquats: scan.aquasecurity[.]org, checkmarx[.]zone, models.LiteLLM[.]cloud
Resource Development	T1583.006	Acquire Infrastructure: Web Services	ICP blockchain canister; Cloudflare Tunnels for relay
Resource Development	T1587.001	Develop Capabilities: Malware	TeamPCP Cloud Stealer, CanisterWorm, Iran wiper, WAV steganography payloads
Initial Access	T1195.002	Supply Chain: Software Supply Chain	Tag poisoning of trivy-action (76 tags), setup-trivy (7), KICS (35), backdoored Trivy binary
Initial Access	T1195.001	Supply Chain: Software Dependencies	CanisterWorm is infecting npm packages consumed as dependencies
Initial Access	T1078.004	Valid Accounts: Cloud Accounts	Compromised aqua-bot, cx-plugins-releases service accounts
Initial Access	T1199	Trusted Relationship	Trivy, Checkmarx, LiteLLM as trusted CI/CD dependencies — the trust relationship is the vector
Execution	T1059.004	Command & Scripting: Unix Shell	Malicious entrypoint.sh (17,592 bytes vs. 2,855 legitimate)
Execution	T1059.006	Command & Scripting: Python	Base64-decoded memory scraping script; sysmon.py/pgmon backdoor
Execution	T1204.002	User Execution: Malicious File	npm postinstall hooks triggering CanisterWorm; .pth auto-execution
Persistence	T1543.002	Systemd Service	sysmon.py dropper, pgmon.service, internal-monitor, pgmonitor, audiomon
Persistence	T1098.001	Account Manipulation: Additional Cloud Credentials	Stolen PyPI/npm tokens used to publish malicious packages
Defence Evasion	T1036.005	Masquerading	PostgreSQL-themed naming (pgmon, pglog); sysmon.py mimics system monitor; msbuild.exe mimics Microsoft Build

Tactic	Technique ID	Technique Name	Campaign Evidence
Defence Evasion	T1036	Impersonation	Imposter commits spoofing rauchg (Vercel CEO) and DmitriyLewen (Trivy maintainer)
Defence Evasion	T1070.004	Indicator Removal: File Deletion	Cleanup after exfiltration; legitimate Trivy scan runs as cover; Telnix Linux variant recursively deletes all artefacts
Defence Evasion	T1480.001	Execution Guardrails	5-min sleep (sandbox evasion); GITHUB_ACTIONS env check; timezone/locale check for Iran targeting
Defence Evasion	T1027.003	Obfuscated Files: Steganography	WAV steganography in Telnix (ringtone.wav, hangup.wav) and K8s wiper (bg_kube.wav, bg_prop.wav)
Credential Access	T1003.007	OS Credential Dumping: /proc	/proc/PID/mem scraping targeting Runner. Worker isSecret:true JSON patterns
Credential Access	T1552.001	Unsecured Credentials: Files	50+ filesystem paths swept for credential files
Credential Access	T1552.004	Unsecured Credentials: Private Keys	SSH, TLS/SSL key harvesting
Credential Access	T1552.005	Steal Cloud Instance Metadata	AWS IMDS querying (IMDSv1 in Checkmarx; full IMDSv2 in LiteLLM)
Collection	T1560.003	Archive via Custom Method	AES-256-CBC + RSA-4096 hybrid encryption → tpcp.tar.gz
Exfiltration	T1041	Exfiltration Over C2 Channel	HTTPS POST to typosquat domains with X-Filename: tpcp.tar.gz
Exfiltration	T1537	Transfer Data to Cloud Account	tpcp-docs/docs-tpcp GitHub repos as fallback exfiltration via release assets
Collection	T1119	Automated Collection	CanisterWorm findNpmTokens() + deploy.js, autonomous harvesting and republishing
Command and Control	T1071.001	Application Layer Protocol: Web	HTTPS to C2 domains
Command and Control	T1090.003	Multi-hop Proxy	Cloudflare Tunnel relay infrastructure
Command and Control	T1571	Non-Standard Protocol	ICP blockchain canister as dead-drop C2 (novel — first documented abuse)
Impact	T1657	Financial Theft	Active extortion using stolen credentials; reported 300 GB credential trove
Impact	T1561.002	Disk Wipe	kamikaze.sh Iran-targeted wiper; rm -rf / on standalone; privileged DaemonSet deletion across K8s nodes
Impact	T1491.002	Defacement: External	44 Aqua Security repos renamed in 79-second scripted burst; made public

Tactic	Technique ID	Technique Name	Campaign Evidence
Impact	T1496	Resource Hijacking	XMRig cryptomining on compromised infrastructure (Operation PCPcat, 60,000+ servers)

This mapping demonstrates coverage across all ten ATT&CK tactics (Reconnaissance through Impact), with particular concentration in Credential Access and Defence Evasion. The ICP blockchain C2 technique (T1571) and the WAV steganography technique (T1027.003) are novel applications not previously documented in supply chain campaigns. The automated collection and republishing via findNpmTokens() (T1119) represents a new class of self-propagating supply chain persistence.

6. Indicators of Compromise (IOCs)

This section consolidates IOCs from vendor advisories (Aqua, Checkmarx, LiteLLM, Telnyx, Docker), security research (Wiz, Datadog, Aikido, JFrog, StepSecurity, Microsoft, SafeDep, Semgrep, Trend Micro), and government advisories (CISA), where a source references additional non-public IOCs (e.g., Trend Micro's full 19-indicator table), which is noted.

6.1 Network, C2, and Exfiltration Infrastructure

Indicator	Type	Context / Phase
scan.aquasecurity[.]org	Domain (typosquat)	Trivy stealer primary exfiltration
45[.]148[.]10[.]212	IP Address	Trivy binary exfiltration
plug-tab-protective-relay.trycloudflare[.]com	Cloudflare Tunnel	Trivy rotating C2
tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0[.]io	ICP Canister	CanisterWorm/Trivy dead-drop C2 (blockchain)
checkmarx[.]zone	Domain (brand impersonation)	Checkmarx phase C2 and exfiltration
checkmarx[.]zone/raw	URL Path	Checkmarx raw payload delivery
checkmarx[.]zone/vsx	URL Path	OpenVSX stolen data exfiltration
models.LiteLLM[.]cloud	Domain (brand impersonation)	LiteLLM phase C2
LiteLLM[.]cloud	Domain	LiteLLM infrastructure
83[.]142[.]209[.]203:8080	IP Address	Telnyx phase C2 / WAV delivery
83[.]142[.]209[.]11	IP Address	Cross-campaign infrastructure (LiteLLM reports)
46[.]151[.]182[.]203	IP Address	Cross-campaign infrastructure
souls-entire-defined-routes.trycloudflare[.]com	Cloudflare Tunnel	CanisterWorm stage payload
investigation-launches-hearings-copying.trycloudflare[.]com	Cloudflare Tunnel	CanisterWorm infrastructure
championships-peoples-point-cassette.trycloudflare[.]com	Cloudflare Tunnel	CanisterWorm infrastructure
hackmoltrepeat[.]com/molt	Domain	hackerbot-claw precursor — token exfiltration
recv.hackmoltrepeat[.]com	Domain	hackerbot-claw precursor — token exfiltration

6.2 File, Service, and Repository Artefacts

Artifact	Context
tcp.tar.gz / X-Filename: tcp.tar.gz	Encrypted exfiltration archive (campaign signature, all phases)
tcp-docs / docs-tcp / tcp-docs-* repo pattern	Fallback exfiltration via GitHub repository creation
~/config/sysmon/sysmon.py	Trivy binary Python dropper
~/config/systemd/user/sysmon.service	Trivy binary systemd persistence
/tmp/runner_collected_*	Trivy GitHub Action temporary exfiltration staging
~/local/share/pgmon/service.py	CanisterWorm Python backdoor
~/config/systemd/user/pgmon.service	CanisterWorm persistence service
/tmp/pglog	CanisterWorm downloaded payload binary
/tmp/pg_state	CanisterWorm payload URL state file
/var/lib/svc_internal/runner.py	Later-stage CanisterWorm/wiper variant
/etc/systemd/system/internal-monitor.service	Later-stage persistence (system-level)
/var/lib/pgmon/pgmon.py	Later-stage pgmonitor variant
/etc/systemd/system/pgmonitor.service	Later-stage persistence variant
LiteLLM_init.pth / LiteLLM_init.pth	LiteLLM auto-execution on Python startup
LiteLLM/proxy/proxy_server.py	LiteLLM backdoored module
ast-results-2.53.0.vsix	Malicious OpenVSX extension (Checkmarx)
cx-dev-assist-1.7.0.vsix	Malicious OpenVSX extension (Checkmarx)
checkmarx-util-1.0.4 / checkmarx-util-1.0.4.tgz	Artefact from checkmarx[.]zone (Trend Micro)
telnyx/_client.py	Telnyx backdoored module
%APPDATA%\...\Startup\msbuild.exe	Telnyx Windows persistent dropper
msbuild.exe.lock	Telnyx Windows lock file
~/config/audiomon/audiomon.py	Telnyx Linux persistence variant
~/config/systemd/user/audiomon.service	Telnyx Linux persistence variant
~/docker/config.json	Target credential file (Docker explicit warning)

6.3 Affected Software Versions and Exposure Windows

Component	Compromised Versions	Safe Version	Exposure Window
aquasecurity/trivy (binary)	v0.69.4, v0.69.5, v0.69.6	v0.69.2, v0.69.3	v0.69.4: ~18:22–21:42 UTC 19 Mar
aquasecurity/trivy-action	76 of 77 tags (v0.0.1–v0.34.2)	@v0.35.0 (SHA: 57a97c7e)	~17:43 19 Mar – ~05:40 20 Mar
aquasecurity/setup-trivy	All 7 tags before the fix	v0.2.6 (rebuilt)	~17:43–21:44 UTC 19 Mar
Docker Hub trivy tags	0.69.4, 0.69.5, 0.69.6, latest	See digests below	19–22 Mar window
LiteLLM (PyPI)	1.82.7, 1.82.8	1.82.6	~10:39 UTC 24 Mar, ~3–5 hrs
telnyx (PyPI)	4.87.1, 4.87.2	4.87.0	03:51–10:13 UTC 27 Mar
Checkmarx/ast-github-action	Before v2.3.33	v2.3.33+	23 Mar
Checkmarx/kics-github-action	Before v5.0.20	v2.1.20+	12:58–16:50 UTC 23 Mar
Checkmarx OpenVSX	ast-results 2.53.0, cx-dev-assist 1.7.0	Prior versions	02:53–15:41 UTC 23 Mar
66+ npm packages	CanisterWorm patch versions	Pre-compromise versions	20–21+ Mar

6.4 Hashes and Digests

6.4.1 Docker Image Digests for Compromised Trivy Tags (Docker official)

Tag	SHA-256 Digest
0.69.4	sha256:27f446230c60bbf0b70e008db798bd4f33b7826f9f76f756606f5417100beef3
0.69.5	sha256:5aaa1d7cfa9ca4649d6ffad165435c519dc836fa6e21b729a2174ad10b057d2b
0.69.6	sha256:425cd3e1a2846ac73944e891250377d2b03653e6f028833e30fc00c1abbc6d33

6.4.2 Malicious Trivy Binary Hashes (Wiz)

Platform	SHA-256
Linux-64bit	822dd269ec10459572dffaefe163dae693c344249a0161953f0d5cdd110bd2a0
Linux-ARM64	e64e152afe2c722d750f10259626f357cdea0420c5eedae37969fbf13abbeef
macOS-ARM64	6328a34b26a63423b555a61f89a6a0525a534e9c88584c815d937910f1ddd538
macOS-64bit	e6310d8a003d7ac101a6b1cd39ff6c6a88ee454b767c1bdce143e04bc1113243
Windows-64bit	0880819ef821cff918960a39c1c1aada55a5593c61c608ea9215da858a86e349
Linux-32bit	f7084b0229dce605ccc5506b14acd4d954a496da4b6134a294844ca8d601970d

Platform	SHA-256
Linux-ARM	bef7e2c5a92c4fa4af17791efc1e46311c0f304796f1172fce192f5efc40f5d7
Linux-PPC64LE	ecce7ae5ffc9f57bb70efd3ea136a2923f701334a8cd47d4fbf01a97fd22859c
Linux-s390x	d5edd791021b966fb6af0ace09319ace7b97d6642363ef27b3d5056ca654a94c
FreeBSD-64bit	887e1f5b5b50162a60bd03b66269e0ae545d0aef0583c1c5b00972152ad7e073

6.4.3 Telnix Package Hashes (SafeDep)

Version	SHA-256
telnix==4.87.1	7321caa303fe96ded0492c747d2f353c4f7d17185656fe292ab0a59e2bd0b8d9
telnix==4.87.2	cd08115806662469bbedec4b03f8427b97c8a4b3bc1442dc18b72b4e19395fe3

6.4.4 CanisterWorm Hashes by Wave (Aikido)

Component	Wave	SHA-256
index.js	Wave 1	e9b1e069efc778c1e77fb3f5fcc3bd3580bbc810604cbf4347897ddb4b8c163b
index.js	Wave 2	61ff00a81b19624adaad425b9129ba2f312f4ab76fb5ddc2c628a5037d31a4ba
index.js	Wave 3	0c0d206d5e68c0cf64d57ffa8bc5b1dad54f2dda52f24e96e02e237498cb9c3a
index.js	Wave 4	c37c0ae9641d2e5329fcdee847a756bf1140fdb7f0b7c78a40fdc39055e7d926
deploy.js	Wave 1	f398f06eefcd3558c38820a397e3193856e4e6e7c67f81ecc8e533275284b152
deploy.js	Wave 2	7df6cef7ab9aae2ea08f2f872f6456b5d51d896ddda907a238cd6668ccdc4bb7
deploy.js	Wave 3+	5e2ba7c4c53fa6e0cef58011acdd50682cf83fb7b989712d2fcf1b5173bad956

6.4.5 Malicious entrypt.sh Hash (CrowdStrike)

File	SHA-256
Malicious entrypt.sh (17,592 bytes)	18a24f83e807479438dcab7a1804c51a00dafc1d526698a66e0640d1e5dd671a
Legitimate entrypt.sh (2,855 bytes)	07500e81693c06ef7ac6bf210cff9c882bcc11db5f16b5bde161218353ba4da

6.4.6 Checkmarx OpenVSX Extension and npm Hashes (SANS / Wiz)

Artifact	SHA-256
ast-results v2.53.0 (.vsix)	65bd72fcdad938cefd55b3323ad29f649a65d4ddd6aeea09afa974dfc7f105d
cx-dev-assist v1.7.0 (.vsix)	744c9d61b66bcd2bb5474d9afeee6c00bb7e0cd32535781da188b80eb59383e0

Artifact	SHA-256
checkmarx-util 1.0.4 (.tgz)	0d66d8c7e02574ff0d3443de0585af19c903d12466d88573ed82ec788655975c

6.5 Kubernetes, Container, and Runtime Behavioural Indicators

Indicator	Type	Source
Pod naming pattern node-setup-*	K8s pod name	Datadog / Aikido
Container names: kamikaze, provisioner	K8s container	Datadog / Aikido
DaemonSet: host-provisioner-iran	K8s DaemonSet	Aikido (Iran wiper)
DaemonSet: host-provisioner-std	K8s DaemonSet	Aikido (non-Iran backdoor)
Outbound SSH with StrictHostKeyChecking=no	SSH lateral movement	Aikido
Outbound connections to port 2375 (Docker API)	Network scan	Aikido
Privileged Alpine containers with hostPath: /	K8s escape	Aikido
pgrep -f Runner.Listener	Process enumeration	Microsoft Defender
pgrep -f Runner.Worker	Process enumeration	Microsoft Defender
kubectl get secrets --all-namespaces -o json true	K8s secret theft	Microsoft Defender
Python-urllib* user agent	Network hunt lead	Microsoft Defender
Outbound HTTP POST with X-Filename: tpcp.tar.gz	Exfiltration header	All phases

7. Current and Potential Impact

7.1 Impact on the Open-Source Ecosystem

The campaign destroys the usual defensive assumption that “official namespace equals safe.” The attacker compromised real projects and real distribution channels. The malicious artefacts were not flagged by conventional vulnerability scanning because they were not vulnerable dependencies — they were legitimate release channels weaponised via stolen publisher credentials.

The mutable Git tag model was a core enabler. Over 10,000 GitHub workflow files reference trivy-action. Every one of them trusted that the tag they pinned still pointed to the code they reviewed. That trust was broken when TeamPCP rewrote 76 of 77 tags in under an hour. The .pth injection technique in LiteLLM v1.82.8 further expanded the attack surface: simply installing the package triggered execution on any Python invocation, even without explicit import.

7.2 Impact on Downstream Users and Commercial Software

Any runner, developer laptop, or build container that executed the poisoned artefacts may have exposed GitHub PATs, cloud service credentials, Kubernetes service account tokens, SSH keys, Docker registry auth, secrets managers, Terraform state, and environment variables. In containerised environments, Docker specifically warns that mounting the Docker socket into a compromised Trivy container can lead to a root-equivalent compromise of the host.

The campaign crossed from OSS into commercial AppSec tooling (Checkmarx), AI infrastructure (LiteLLM, which serves ~95 million monthly downloads and typically stores multiple LLM provider API keys), and telecommunications SDKs (Telnyx, with ~670,000 monthly downloads). This demonstrates a “downstream signing and trust amplification” problem: a compromised CI/CD runner can leak publisher keys that are then used to ship malicious updates from real vendor accounts into production channels.

7.3 Cache Persistence Risk

A secondary but operationally critical risk is cache persistence. Even after malicious upstream content is removed from registries, artefact proxy caches (Nexus, Artifactory), registry mirrors, Docker local stores, CI worker images, golden base images, and developer laptop caches may still hold the compromised bytes. Organisations must actively search these caches for the specific compromised digests and versions, rather than assuming that upstream remediation automatically propagates.

8. Impact Scoping: Comparison with SolarWinds and Log4j

This campaign should not be framed as “another SolarWinds” or “another Log4j.” It sits between them. It has a narrower software footprint than Log4j because it does not affect a near-universal library embedded across vast numbers of Java products. It is also less clearly strategic and less stealth-proven than SolarWinds, where trojanised Orion updates reached roughly 18,000 customers and a smaller, high-value subset was then selectively exploited for espionage. But it is still a serious event because it compromises trust anchors in modern software delivery: CI/CD actions, release automation, package publishing, and security tooling itself.

8.1 Compared with SolarWinds

The SolarWinds campaign compromised Orion, a network management platform deployed deep inside enterprise and government environments, where it naturally had privileged visibility into infrastructure. U.S. sources later assessed that while up to 18,000 customers received the tainted update, fewer than 100 were actually compromised through SUNBURST, underscoring that SolarWinds combined broad initial seeding with selective follow-on exploitation for espionage.

By contrast, the current TeamPCP wave is centred on developer ecosystems and build systems: malicious Trivy releases, poisoned GitHub Actions, compromised PyPI and npm packages, and downstream abuse of publishing credentials. That makes it less like a covert strategic implant in network management infrastructure, and more like a developer-trust compromise designed to steal secrets and pivot fast across the software supply chain. TeamPCP currently appears less optimised for long-dwell espionage inside national-level critical networks than SolarWinds was.

8.2 Compared with Log4j

NCSC described Log4Shell as affecting a widely used logging tool used by millions of computers worldwide, while CIS estimated the flaw was present in over 100 million instances globally. CISA and its partners described the exploitation of Log4j as active and widespread across consumer, enterprise, and even operational technology products. That is a very different exposure model from TeamPCP, where compromise depends on whether an organisation pulled a specific malicious version, referenced a poisoned mutable tag, or consumed a compromised package during a finite exposure window. In terms of sheer breadth, Log4j remains the larger class of events.

8.3 Why TeamPCP Is More Dangerous Than a Normal “Bad Package” Incident

TeamPCP is more dangerous than a normal supply chain incident in one specific way: it attacks the security and release machinery that other software depends on. Microsoft describes the Trivy phase as a CI/CD-focused supply-chain attack that simultaneously compromised the scanner binary and its associated GitHub Actions. At the same time, NVD notes that the actor used compromised credentials, force-pushed 76 of 77 trivy-action tags, and replaced all 7 setup-trivy tags with malicious commits. The campaign has since expanded to affect LiteLLM, Checkmarx, multiple npm namespaces, and Telnyx, demonstrating the ability to cascade across ecosystems. While it is not as universal as Log4j, it is more structurally dangerous than a single vulnerable library because it can turn trusted build paths into malware distribution paths.

This creates what researchers have described as a trust inversion problem: the organisations most diligent about running security scans in their build pipelines had the greatest attack surface

exposure. The security tool itself was the weapon. Security tooling supply chains, therefore require stricter controls than application supply chains, because compromise can deliver attacker code into the most privileged positions in the development lifecycle.

8.4 Systemic Architectural Failures That Enabled the Campaign

Dan Lorenc, CEO of Chainguard and former Google supply chain security lead, catalogued six specific architectural failures in GitHub Actions that enabled this campaign:

- **Mutable tags without transparency logs:** Git tags are mutable pointers that anyone with push access can silently redirect. There is no transparency log recording when a tag is modified. This is the mechanism that allowed 76 tags to be force-pushed to malicious commits without alerting maintainers or consumers.
- **Overpermissive default GITHUB_TOKEN:** The default token scope grants more access than most workflows need, expanding the blast radius when compromised.
- **The dangerous pull_request_target trigger:** Grants fork workflows access to base repository secrets by design. This was the initial access vector for hackerbot-claw and is the same anti-pattern exploited in the 2024 SpotBugs breach and the 2025 Nx/s1ngularity attack.
- **Unsafe template syntax:** GitHub Actions template expressions drop user-controlled values into shell scripts without escaping, enabling injection attacks.
- **Transitive dependency gaps:** SHA pinning protects the direct action reference, but composite actions or components within them may reference other actions via mutable tags, undermining pinning one level down.
- **No publication controls for Actions:** There is no equivalent of code signing, provenance attestation, or approval gating for publishing or modifying GitHub Actions in the public marketplace.

Lorenc's assessment is blunt: the design of GitHub Actions is "plain irresponsible today and ignores a decade of supply chain security work from other ecosystems." Chainguard's research also demonstrated that even GitHub's organisation-level allowlist for permitted actions can be bypassed by imposter commits from forked repositories, undermining the very mitigation that organisations are advised to implement. These are architectural design decisions, not bugs. GitHub has only partially addressed them: Immutable Releases were introduced post-attack, but they do not protect non-release tags.

8.5 Highest-Risk Victims

The highest-risk victims are not general end users. They are: upstream OSS maintainers and small maintainer teams; commercial software vendors whose developers or pipelines pulled affected artefacts; CI/CD administrators; developers with publish rights to PyPI, npm, Docker Hub, or OpenVSX; AI and platform teams using packages such as LiteLLM; and any organisation that builds, signs, scans, or redistributes software to others. In other words, the danger is concentrated on the people and systems that can amplify compromise downstream. That makes this campaign especially relevant to software vendors, MSSPs, system integrators, government contractors, and platform engineering teams, even when they are not themselves the original intended target.

8.6 Trivy Is Not Confined to Dev/UAT Environments

It would be too optimistic to characterise Trivy as a tool used only in development or UAT environments. Official Trivy documentation shows it can scan Kubernetes clusters directly. It can be installed as a Kubernetes Operator that continuously scans cluster resources (the Trivy Operator tutorial documents automatic scanning every six hours). Docker further warns that if a compromised Trivy image ran with the Docker socket mounted, the container effectively had root-level access to the host. The campaign is therefore not confined to developer laptops and test environments. It can also reach build runners, registry mirrors, artefact caches, cluster-side scanners, and container hosts, which are often much closer to production trust boundaries than a normal development workstation.

8.7 Direct vs Indirect Production Impact

For critical infrastructure and enterprise production environments, the most defensible judgment is that direct production infection is likely lower than with Log4j and probably lower than with SolarWinds, but indirect production impact is not low. The direct path is narrower because this campaign does not automatically expose every internet-facing Java service, and Trivy itself is not as deeply embedded in mainstream production application stacks as Log4j was. However, if a compromised build environment exposed cloud credentials, Kubernetes tokens, SSH keys, or registry credentials, the attacker may not need the package to exist in production at all. They can move from dev or CI into artefact repositories, cloud control planes, container registries, signing workflows, or deployment secrets.

CRITICAL SCOPING LANGUAGE

The right scoping language is NOT “production impact is limited.”

The stronger and more accurate wording is: “Initial compromise is concentrated in software development and build environments, but downstream production impact depends on the privilege and connectivity of those environments.”

Where CI/CD systems hold production cloud credentials, deployment tokens, registry push rights, or signing keys, the campaign can absolutely become a production incident even if the malicious scanner never runs on a production server.

Where build systems are isolated, use short-lived credentials, and are separated from production deployment authority, the blast radius is materially reduced.

NVD explicitly states that if compromised versions run, all secrets accessible to those pipelines should be treated as exposed and rotated immediately.

8.8 Bottom Line: Impact Scoping

SolarWinds was a stealthy, selective espionage operation seeded through a highly privileged monitoring platform. Log4j was a near-universal internet-scale software flaw that was hard to enumerate because it was buried deep in transitive dependencies. TeamPCP is neither of those exactly. It is a developer-ecosystem supply-chain compromise: narrower than Log4j in ubiquity, less strategically proven than SolarWinds in stealth and critical infrastructure penetration, but potentially devastating to organisations whose build and release systems hold powerful secrets or publish software to others.

FOR MOST ENTERPRISES

The primary risk is NOT mass production exploitation on day one.

It IS secret theft in CI/CD followed by downstream compromise of software, cloud, and deployment trust.

For critical infrastructure specifically, immediate direct exposure may be limited if affected artefacts were not used in production-facing paths. However, organisations with layered DevSecOps controls should not assume immunity — edge exposure through developer workstations, cached artefacts, contractor pipelines, or independently managed environments remains a residual risk that must be actively hunted.

9. Supply Chain Attack Lineage: TeamPCP in Context

The TeamPCP campaign did not emerge in isolation. It is the latest and most severe in an accelerating series of CI/CD and package registry supply chain attacks that share common architectural vulnerabilities, escalating techniques, and in some cases direct credential lineage. Understanding this lineage is essential for anticipating what comes next.

9.1 Evolution Comparison

Dimension	tj-actions / SpotBugs (Mar 2025)	s1ngularity / Nx (Aug 2025)	Shai-Hulud 1.0 & 2.0 (Sep–Nov 2025)	TeamPCP (Feb–Mar 2026)
CVE / Tracking	CVE-2025-30066, CVE-2025-30154	GHSA-cxm3-wv7p-598c	No CVE assigned	CVE-2026-33634 (CVSS 9.4)
Initial Access	Stolen SpotBugs maintainer PAT → reviewdog → tj-actions (4-hop chain starting Nov 2024)	Exploited pull_request_target in the Nx GitHub Actions workflow	Compromised npm credentials (possibly from s1ngularity credential pool)	AI-powered hackerbot-claw exploited pull_request_target in Trivy (Feb 2026)
Primary Target	Coinbase (specifically agentkit repo); broadened after detection	Nx build system (3.5M weekly downloads); developer workstations	npm ecosystem broadly; major projects (Zapier, PostHog, Postman, ENS Domains)	Trivy → Checkmarx → LiteLLM → Telnix; security/AI tooling specifically
Ecosystems Hit	GitHub Actions only	npm, GitHub repos	npm (700+ packages), GitHub repos (25,000+)	GitHub Actions, Docker Hub, npm (66+), OpenVSX, PyPI — 5 ecosystems
Self-Propagation	Limited (PAT chain, 4 hops over months)	No (single-burst credential harvest)	Yes — first npm worm; findNpmTokens() + auto-republish; “Shai-Hulud” repo creation	Yes — CanisterWorm (evolved from Shai-Hulud pattern); deploy.js auto-publishes across scopes
C2 Infrastructure	GitHub workflow logs (secrets printed to logs)	GitHub repos (s1ngularity-repository pattern, double-base64)	GitHub repos (Shai-Hulud marker); self-hosted runner registration as SHA1HULUD	ICP blockchain canister (novel, takedown-resistant); vendor-specific typosquat domains
Persistence	None (secrets in logs)	Shell startup modification (sudo shutdown); short-lived	Self-hosted runner registration; pre-install hooks; dead man’s switch data wiper	systemd user services (sysmon, pgmon, audiomon); .pth auto-execution; K8s DaemonSets
Stealth	High (3-month dormancy before execution; initially targeted at	Low (visible repo defacement; shutdown commands)	Medium (Bun runtime obfuscation in v2; pre-install timing; but visible Shai-Hulud markers)	High (legitimate scan runs in parallel; vendor-specific typosquat C2; encrypted exfil; artifact cleanup)

Dimension	tj-actions / SpotBugs (Mar 2025)	s1ngularity / Nx (Aug 2025)	Shai-Hulud 1.0 & 2.0 (Sep–Nov 2025)	TeamPCP (Feb–Mar 2026)
	Coinbase surgically)			
Destructive Capability	None	Shell startup sabotage (sudo shutdown)	Dead man's switch: home directory wipe if propagation fails	Iran-targeted K8s cluster wiper (kamikaze.sh); rm -rf / on standalone Iranian hosts
AI Usage	None documented	First to weaponise AI CLI tools (Claude, Gemini, Q) for reconnaissance	LLM-generated malicious scripts (assessed with moderate confidence by Unit 42)	AI-powered initial access agent (hackerbot-claw, self-described as claude-opus-4-5 powered)
Credential Mutualization	4-hop chain but slow (months)	Single burst; credentials reused in second wave	Yes — stolen tokens from one victim used to create exfil repos and propagate to other victims; token-sharing botnet in v2	Yes — each phase's stolen credentials unlock the next; 5 ecosystems in 9 days
Scale	23,000 repos affected; 218 confirmed secret exposures	1,079 systems; 2,349 credentials; 10,767 private repos made public	700+ npm packages; 25,000+ malicious repos; 33,185 secrets exposed (3,760 valid)	1,000+ SaaS environments (Mandiant); 66+ npm packages; 141 malicious artifacts; 300 GB compressed credentials
Attribution	Unattributed (assessed as distinct from TeamPCP by SANS)	UNC6426 (Google/Mandiant tracking); unconfirmed link to Shai-Hulud	Unattributed (possible link to s1ngularity via credential pool)	TeamPCP (DeadCatx3/PCPcat/Persy PCP/ShellForce); high-confidence attribution
Extortion	None	None (but AWS admin access achieved in 72 hours per Google)	None documented	Active extortion with reported LAPSUS\$ collaboration (UNVERIFIED)

9.2 The Credential Lineage Chain

A critical observation across this attack lineage is the possibility of credential mutualisation — where credentials stolen in one campaign seed the next. The chain is not definitively proven end-to-end, but the circumstantial evidence is strong:

- **s1ngularity → Shai-Hulud:** GitGuardian and Dark Reading documented that all Shai-Hulud 1.0 patient-zero packages had Nx as a dependency, suggesting that npm tokens stolen during the s1ngularity attack in August 2025 may have enabled the September Shai-Hulud campaign. The connection is assessed as probable but not forensically confirmed.
- **Shai-Hulud → TeamPCP:** The Shai-Hulud 2.0 campaign (November 2025) exposed 33,185 secrets, including 3,760 that remained valid. TeamPCP's Operation PCPcat (December 2025) compromised 60,000+ servers. Whether TeamPCP acquired

credentials from the Shai-Hulud pool or independently developed parallel capabilities is unknown. What is clear is that both groups evolved the same core technique: npm worm propagation via stolen tokens with auto-republish.

- **TeamPCP internal chaining (proven):** Within the March 2026 campaign, the credential chain is well-documented: Trivy credentials (19 Mar) → Checkmarx credentials (23 Mar) → LiteLLM PyPI token (24 Mar) → likely Telnyx PyPI token (27 Mar). Each phase's stolen credentials unlocked the next, crossing ecosystem boundaries in the process.

9.3 Technique Evolution: What Each Campaign Learned

9.3.1 tj-actions/SpotBugs (March 2025): The Proof of Concept

The tj-actions campaign proved that GitHub Actions tag poisoning could cascade across vendor boundaries. The attacker traced a 4-hop chain (SpotBugs maintainer PAT → reviewdog/action-setup → tj-actions/eslint-changed-files → tj-actions/changed-files) over several months, initially targeting Coinbase with surgical precision. When the Coinbase attack was detected, the attacker pivoted to a broad, indiscriminate attack affecting 23,000+ repositories. Unit 42 traced the origin to a stolen SpotBugs PAT and a disposable GitHub account (jurkaofavak) that was added as a member and deleted within minutes. CISA added CVE-2025-30066 to the KEV catalogue. This campaign demonstrated that a single stolen PAT could compromise an entire dependency tree through mutable tag manipulation — the exact technique TeamPCP would later use at scale.

9.3.2 s1ngularity / Nx (August 2025): AI-Assisted Reconnaissance

The s1ngularity attack targeted the Nx build system (3.5 million weekly npm downloads) by exploiting the same pull_request_target + unsanitised-input vulnerability class. What made it novel was its weaponisation of AI developer tools: the malicious telemetry.js script checked whether Claude Code, Gemini CLI, or Amazon Q was installed on the victim's machine and, if so, used them to aid in reconnaissance and credential discovery. StepSecurity documented this as the first known case of malware weaponising AI CLI tools. Wiz identified over 1,000 valid GitHub tokens and dozens of cloud credentials among the exfiltrated data. Google later tracked the threat actor as UNC6426, documenting a case where stolen credentials were escalated to full AWS AdministratorAccess within 72 hours via a CloudFormation stack deployment. The s1ngularity campaign was live for approximately 5 hours before it was detected, but compromised 1,079 developer systems.

9.3.3 Shai-Hulud 1.0 and 2.0 (September–November 2025): The First npm Worm

Shai-Hulud 1.0 (September 2025) was the first true self-propagating worm in the npm ecosystem. It automated the full credential-to-compromise cycle: installing a compromised package, harvesting npm tokens, enumerating publishable packages, injecting the malicious payload, and republishing. GitHub reported it as a “coordinated, multi-wave campaign that evolved from opportunistic compromises to engineered, targeted attacks.” The first known compromised package was @ctrl/tinycolor v4.1.1.

Shai-Hulud 2.0 (“The Second Coming,” November 2025) escalated dramatically. Wiz documented over 25,000 malicious repositories across ~500 GitHub users and ~700 compromised npm packages, including major projects from Zapier, ENS Domains, PostHog, and Postman. GitGuardian analysed the exfiltrated data and found 33,185 unique secrets exposed, of which 3,760 remained valid days after discovery. Key technical innovations included: execution during preinstall (not postinstall) to widen the area of impact; use of the Bun

runtime to hide core malicious logic; a self-healing botnet inside GitHub where compromised machines shared stolen tokens; registration of compromised hosts as self-hosted GitHub Actions runners (named SHA1HULUD) for persistent remote code execution via GitHub Discussions; and a dead man's switch that attempted to destroy the victim's home directory if propagation failed. Microsoft published dedicated Defender guidance for the campaign.

9.3.4 TeamPCP (February–March 2026): The Convergence

TeamPCP represents the convergence of every technique pioneered in the preceding campaigns, executed with greater sophistication and cross-ecosystem reach:

- **From tj-actions:** GitHub Actions tag poisoning via stolen PATs, mutable tag manipulation, and imposter commits spoofing maintainer identities.
- **From s1ngularity:** AI-assisted initial access (hackerbot-claw vs s1ngularity's AI CLI tool abuse), pull_request_target exploitation, and the concept of weaponising developer tooling.
- **From Shai-Hulud:** Self-propagating npm worm (CanisterWorm is a direct technical descendant of Shai-Hulud's propagation model), credential mutualisation across victims, and GitHub repo creation as exfiltration fallback (tpcp-docs mirrors Shai-Hulud's marker repos).
- **TeamPCP's innovations beyond predecessors:** ICP blockchain C2 (replacing Shai-Hulud's GitHub-based C2 with takedown-resistant infrastructure); cross-ecosystem pivoting across 5 ecosystems (predecessors stayed within npm/GitHub); targeting of security tooling itself (Trivy, Checkmarx) rather than generic packages; WAV steganography for payload delivery; encrypted exfiltration with AES/RSA (predecessors used plaintext or base64 encoding); and the geopolitically-targeted Iran wiper.

9.4 The Acceleration Pattern

The cadence of these attacks is accelerating. The SolarWinds campaign (2020) and Codecov (2021) were one-hop attacks separated by a year. The tj-actions campaign (March 2025) introduced multi-hop credential chaining. Within six months, s1ngularity (August 2025) added AI-assisted reconnaissance. Two months later, Shai-Hulud (September 2025) introduced self-propagating worm behaviour. Its second wave (November 2025) added destructive capabilities. Four months later, TeamPCP (March 2026) combined all of these techniques into a single campaign spanning five ecosystems. The interval between major campaigns is shrinking from years to months, and the sophistication of each successor exceeds its predecessor.

WHAT COMES NEXT

The structural conditions now exist for supply chain attacks at this scale to happen every month (Endor Labs). Each campaign teaches the next attacker: tj-actions proved that tag poisoning works; s1ngularity proved that AI tools can be weaponised; Shai-Hulud proved that worms can propagate autonomously; TeamPCP proved that the entire model works across multiple ecosystems simultaneously.

Anticipated next-generation capabilities include: self-propagating worms in PyPI, RubyGems, crates.io, and Maven Central (TeamPCP has the tooling, Endor Labs projects adaptation is trivial); LLM-generated exploit code at scale (the CVE-GENIE framework demonstrated 51% exploit reproduction at \$2.77 per vulnerability); blockchain C2 adoption by additional threat actors now that ICP canister effectiveness is proven; and credential mutualization marketplaces where stolen tokens from one campaign are sold or traded to seed the next.

The uncomfortable truth: the mitigations recommended throughout this report — pin to commit SHAs, restrict allowed actions, monitor CI/CD egress — are workarounds for architectural decisions that should never have shipped as defaults.

10. Threat Actor Profile and Assessed Motives

Attribute	Assessment
Designation	TeamPCP (aliases: DeadCatx3, PCPcat, PersyPCP, ShellForce, CipherForce)
Activity Period	December 2025 – Present (Active)
Classification	Cloud-native cybercriminal operation; hybrid model combining credential theft, access brokering, cryptomining, and destructive operations
Assessed Collaboration	UNVERIFIED: Secondary reporting speculates links to LAPSUS\$ (extortion) and Vect (ransomware). No primary-source confirmation identified as of 28 March 2026.
Communication	Telegram channels @Persy_PCP and @teampcp; public taunting of victims; campaign branding embedded in payloads
Attribution Confidence	HIGH (consistent RSA key pair, tpcp.tar.gz naming, tpcp-docs repo pattern, TeamPCP Cloud stealer string in payloads — per Snyk, Wiz, Datadog, Endor Labs)
Nationality / Location	Unconfirmed. Timestamp analysis (SANS/Boost Security) shows two activity clusters: 02:00–06:00 UTC and 17:00–19:00 UTC, consistent with the Americas timezone UTC-4 to UTC-9. No Russian, Chinese, Farsi, or other non-English indicators in Telegram posts, code comments, or communications. No presence on Russian-language criminal forums. Assessment: likely a small team of 2–4 operators based in the Western hemisphere.
C2 Hosting Infrastructure	Primary C2 is hosted on TECHOFF SRV LIMITED (ASN AS48090), a UK-registered company identified by Team Cymru as a bulletproof hosting facade, with a registered address at 35 Firs Avenue, London N11 3NE, shared with 1,000+ companies. 236 domains across 57 IPs in Amsterdam. scan.aquasecurity[.]org resolves to 45.148.10.212 within this allocation. Additional infrastructure includes Sliver C2 staging servers (67.217.57.240, 44.252.85.168) and five Cloudflare Quick Tunnels.
Operational Style	“Strength does not come from novel exploits or original malware, but from the large-scale automation and integration of well-known attack techniques” (Flare Research). Ego-driven defacement, Rick Roll payloads, and performative branding suggest skilled but undisciplined operators building a criminal reputation.
AI Capabilities	Uses hackerbot-claw, an AI agent built on openclaw, for automated vulnerability scanning and initial access across multiple OSS repositories
Novel Techniques	First ICP blockchain canister as C2; first self-propagating npm worm; WAV steganography; AI-powered autonomous initial access
Known Targeting (General)	UAE, Canada, South Korea, Serbia, US, Vietnam — opportunistic cloud infrastructure exploitation
Known Targeting (Destructive)	Iran — geopolitically targeted Kubernetes wiper (Asia/Tehran timezone, fa_IR locale)
Industry Focus	BFSI, Consumer Goods, Professional Services, DevOps/Security tooling vendors, AI/ML infrastructure

10.1 Assessed Motives

The motive profile is assessed as primarily credential theft, access expansion, and monetisable downstream compromise:

- **Credential Theft and Access Brokering:** The systematic harvesting of cloud credentials, API keys, publishing tokens, and Kubernetes access across multiple ecosystems suggests an access-brokering model where harvested credentials are sold, traded, or reused for follow-on intrusion.
- **Financial Gain:** Cryptomining (XMRig deployment on compromised hosts), and potential ransomware operations (if collaboration claims are confirmed).
- **Reputation and Signalling:** Public taunting, embedding campaign branding in payloads, defacement of compromised repositories, and suppression of community reporting all indicate a desire for recognition.
- **Geopolitical Targeting:** The Iran-targeted Kubernetes wiper introduces a geopolitical dimension. Whether this reflects state alignment, personal ideology, or client-directed operations remains under assessment.

11. Recommended Course of Actions

11.1 Immediate Actions (0–48 Hours)

- **Treat this as a credential exposure campaign first, package incident second.** Any environment that executed the named bad versions should have ALL reachable secrets rotated: GitHub PATs, cloud credentials (AWS, GCP, Azure), Kubernetes tokens, SSH keys, Docker registry auth, npm/PyPI publishing tokens, Terraform state, and environment variables. Rotation must be atomic.
- **Scope by exact artefacts and time windows.** Hunt for Trivy v0.69.4–0.69.6; trivy-action before v0.35.0; setup-trivy before v0.2.6; Docker tags 0.69.4/0.69.5/0.69.6/latest; LiteLLM 1.82.7/1.82.8; Checkmarx affected actions/plugins; Telnyx 4.87.1/4.87.2; and the named npm namespaces. Use the precise exposure windows in Section 6.3.
- **Deploy IOCs from Section 6 to SIEM and network monitoring.** Block all identified C2 domains, IP addresses, and ICP canister endpoints. Monitor for tcp.tar.gz exfiltration headers, ICP beaconing to icp0.io, and outbound connections to the listed infrastructure.
- **Search caches and mirrors.** Check Nexus, Artifactory, registry mirrors, Docker caches, CI worker images, golden base images, and developer laptop caches for compromised digests (Section 6.4) or package versions. Upstream remediation does NOT automatically propagate to caches.
- **Search GitHub organisations for tcp-docs / docs-tcp repositories.** Their presence indicates successful credential exfiltration via the fallback mechanism.
- **On Linux systems, hunt for CanisterWorm persistence.** Check for pgmon.service, pgmonitor.service, internal-monitor.service, audiomon.service in the systemd user and system service directories. Check /tmp/pglog, /tmp/pg_state, ~/.local/share/pgmon/.

11.2 Short-Term Actions (1–2 Weeks)

- **Pin all GitHub Actions to full, immutable commit SHA hashes.** Mutable version tags must not be used. This is the single most effective control against tag-poisoning attacks.
- **Pin container images to digests plus provenance verification.** Tag-based pulls are vulnerable to the same class of attack.
- **Pin Python/npm dependencies to vetted versions with hash verification.** Use pip --require-hashes, package-lock.json with integrity fields, and verify dependency integrity before installation.
- **Disable npm postinstall hooks by default.** Use npm config set ignore-scripts true globally. Selectively enable with @lavamoat/allow-scripts.
- **Conduct a full dependency audit** across all applications, including transitive dependencies. Pay particular attention to .pth files and postinstall hooks.
- **Restrict CI/CD runner network egress.** Implement allowlist-based outbound networking. Block IMDS or enforce IMDSv2 with hop limits.
- **Reduce secret blast radius in CI/CD.** Minimise default token scopes, use short-lived credentials, isolate release credentials from scan credentials, and make secret rotation atomic during incident response.
- **Enable runtime detection on CI runners.** Deploy Sysdig Secure, Falco, or equivalent to detect credential theft and data exfiltration from build environments.

11.3 Medium-Term Actions (1–3 Months)

- **Adopt SLSA Level 3+** across all critical CI/CD pipelines. Ensure provenance metadata is generated and verified for all artefacts.
- **Implement Trusted Publisher for PyPI** (OIDC-based authentication from CI/CD). Eliminate long-lived PyPI tokens.
- **Deploy ICP/blockchain C2 detection rules.** Monitor for DNS queries and connections to icp0.io domains. This technique is expected to be adopted by additional threat actors.
- **Instrument runtime and network detections.** Monitor for runner memory scraping (pgrep -f Runner.Worker), abnormal outbound POSTs from scanners, unexpected repo creation, unauthorised publishes, suspicious OpenVSX plugin installs, and privileged K8s DaemonSet creation.
- **Establish a dependency allowlisting framework—Centralise** vetting and approval of third-party OSS dependencies.
- **Conduct tabletop exercises** using the TeamPCP TTP playbook (credential chaining, tag poisoning, self-propagating worms, cache persistence) as the threat model.
- **Monitor Certificate Transparency logs for pre-operation indicators.** Analysis of this campaign revealed a consistent operational pattern: TeamPCP provisioned TLS certificates 1–2 days before each attack phase. CT logs recorded scan.aquasecurity[.]org on 17 March (2 days before the Trivy attack), checkmarx[.]zone on 22 March (1 day before Checkmarx), and models.LiteLLM[.]cloud on 23 March (1 day before LiteLLM). Organisations can monitor CT logs via crt.sh, Certstream, or commercial CT monitoring for typosquat domains mimicking their brand or their security vendors' brands. This is a detectable pre-operation indicator. (Source: SANS Institute, Hartman & Johnson.)
- **Require signed commits and configure tag protection rules via GitHub rulesets.** The imposter commit technique used in this campaign — spoofing rauchg (Vercel CEO) and Dmitry Lewen (Trivy maintainer) as commit authors — is defeated by enforced commit signing. Git author/committer fields are unauthenticated metadata that can be set to any value; GitHub displays the spoofed identity's avatar, lending it false credibility. Use GitHub rulesets (not just branch protection) to enforce signature requirements across both branches and tags. For tag protection specifically: restrict who can create, update, or delete tags; apply tag name patterns (e.g., v*) to protect version tags; require signature verification on tag operations; and monitor tag mutation events. Tag deletion plus recreation with the same name is inherently suspicious. The force-pushing of 76 tags to malicious commits exploited the fact that tags have no protection by default.
- **Distinguish incident-response rotation from routine rotation.** Aqua Security's credential rotation after the initial breach followed a routine zero-downtime pattern (issue new → update → verify → revoke old). During an active compromise, the attacker can observe new credentials deployed through the same infrastructure they already control. Incident-response rotation requires: (1) immediately revoke all compromised credentials to cut off access — accept the service disruption; (2) generate new credentials through an out-of-band channel the attacker cannot observe (separate admin console, clean workstation, not the compromised CI/CD pipeline); (3) deploy new credentials only after confirming the attacker's access path has been eliminated; (4) monitor for authentication attempts using revoked credentials. This distinction is why the February rotation failed, and the March 19 attack was possible.
- **Restrict which GitHub Actions your organisation can use.** Navigate to Organisation Settings → Actions → General, select "Allow select actions and reusable workflows,"

and curate a list of vetted actions pinned to specific commit SHAs. Require a security review before adding new actions. Be aware that Chainguard's research demonstrated that imposter commits can bypass this allowlist — so combine it with commit-signing enforcement.

12. Summary

This campaign is one of the clearest recent examples of how a compromise of trusted developer tooling can become a cross-ecosystem operation for credential theft and malware distribution. It started with Trivy, then spread into GitHub Actions, Docker Hub, npm, Checkmarx/OpenVSX, LiteLLM on PyPI, and Telnix. The technical common thread is stolen trust: publisher credentials, bot accounts, mutable tags, CI secrets, and package-manager trust models.

For any organisation, the immediate priority is not only “do we use package X?” It is “did any trusted build, developer, or release environment execute one of these poisoned artefacts, and if so, what secrets, tokens, and downstream software releases were exposed afterwards?” That is the right lens for security teams, platform engineers, and executive leadership.

The fundamental lesson is that the open-source software supply chain is now a primary attack surface — not a peripheral risk. Organisations must treat dependency management with the same rigour applied to their own source code, pin dependencies to immutable references, actively search caches and mirrors for compromised artefacts, monitor for anomalous behaviour in build environments, and maintain rapid, atomic credential rotation capabilities.

CAMPAIGN STATUS AS OF 28 MARCH 2026

This campaign is **ACTIVE** and **ONGOING**. TeamPCP continues to expand to new targets and ecosystems. Additional compromises across PyPI, RubyGems, crates.io, and Maven Central are anticipated.

Cache persistence means that even “remediated” environments may still hold compromised artefacts. Active hunting is required.

All IOCs in this report should be treated as **ACTIVE**. Monitor cited sources for updates.

13. References

Sources are grouped by type. Primary vendor advisories are listed first, followed by government advisories, then security research. All URLs accessed on or before 28 March 2026.

#	Source	Title	URL
1	Aqua Security	Trivy Supply Chain Attack — What You Need to Know	https://www.aquasec.com/blog/trivy-supply-chain-attack-what-you-need-to-know/
2	Aqua Security / GitHub	Trivy Security Incident 2026-03-19 Discussion #10425	https://github.com/aquasecurity/trivy/discussions/10425
3	GitHub Advisory	GHSA-69fq-xp46-6x23 — Trivy Ecosystem Supply Chain Compromise	https://github.com/aquasecurity/trivy/security/advisories/GHSA-69fq-xp46-6x23
4	Docker	Trivy Supply Chain Compromise: What Docker Hub Users Should Know	https://www.docker.com/blog/trivy-supply-chain-compromise-what-docker-hub-users-should-know/
5	Checkmarx	Checkmarx Security Update	https://checkmarx.com/blog/checkmarx-security-update/
6	LiteLLM	Security Update March 2026	https://docs.litellm.ai/blog/security-update-march-2026
7	Telnyx	Python SDK Supply Chain Security Notice March 2026	https://telnyx.com/resources/telnyx-python-sdk-supply-chain-security-notice-march-2026
8	CISA	Adds CVE-2026-33634 to the Known Exploited Vulnerabilities Catalogue	https://www.cisa.gov/news-events/alerts/2026/03/26/cisa-adds-one-known-exploited-vulnerability-catalog
9	NVD	CVE-2026-33634 Detail	https://nvd.nist.gov/vuln/detail/CVE-2026-33634
10	Wiz	Trivy Compromised by TeamPCP — Supply Chain Attack	https://www.wiz.io/blog/trivy-compromised-teampcp-supply-chain-attack
11	Datadog Security Labs	LiteLLM and Telnyx Compromised on PyPI: Tracing the TeamPCP Campaign	https://securitylabs.datadoghq.com/articles/LiteLLM-compromised-pypi-teampcp-supply-chain-campaign/
12	Sysdig TRT	TeamPCP Expands: Supply Chain Compromise Spreads from Trivy to Checkmarx	https://www.sysdig.com/blog/teampcp-expands-supply-chain-compromise-spreads-from-trivy-to-checkmarx-github-actions
13	Microsoft Security Blog	Detecting, Investigating, and Defending Against the Trivy Supply Chain Compromise	https://www.microsoft.com/en-us/security/blog/2026/03/24/detecting-investigating-defending-against-trivy-supply-chain-compromise/
14	Snyk	How a Poisoned Security Scanner Became the Key to Backdooring LiteLLM	https://snyk.io/articles/poisoned-security-scanner-backdooring-LiteLLM/
15	Aikido Security	TeamPCP Deploys CanisterWorm on NPM Following Trivy Compromise	https://www.aikido.dev/blog/teampcp-deploys-worm-npm-trivy-compromise
16	Aikido Security	TeamPCP Stage Payload CanisterWorm Iran	https://www.aikido.dev/blog/teampcp-stage-payload-canisterworm-iran

#	Source	Title	URL
17	Aikido Security	Telnyx PyPI Compromised by TeamPCP CanisterWorm	https://www.aikido.dev/blog/telnyx-pypi-compromised-teampcp-canisterworm
18	JFrog Security Research	CanisterWorm — New Compromised Versions Detected	https://research.jfrog.com/post/canister-worm/
19	JFrog Security Research	TeamPCP Strikes Again — Telnyx Popular Library Hit	https://research.jfrog.com/post/team-pcp-strikes-again-telnyx-popular-library-hit/
20	Socket	CanisterWorm: npm Publisher Compromise	https://socket.dev/blog/canisterworm-npm-publisher-compromise-deploys-backdoor-across-29-packages
21	Mend.io	CanisterWorm: The Self-Spreading npm Attack	https://www.mend.io/blog/canisterworm-the-self-spreading-npm-attack-that-uses-a-decentralized-server-to-stay-alive/
22	Endor Labs	TeamPCP Isn't Done: LiteLLM's 95M Monthly Downloads	https://www.endorlabs.com/learn/teampcp-isnt-done
23	SafeDep	Malicious Telnyx PyPI Compromise	https://safedep.io/malicious-telnyx-pypi-compromise
24	Semgrep	TeamPCP Credential Infostealer Chain Attack Reaches LiteLLM	https://semgrep.dev/blog/2026/the-teampcp-credential-infostealer-chain-attack-reaches-pythons-LiteLLM/
25	Trend Micro	Inside the LiteLLM Supply Chain Compromise	https://www.trendmicro.com/en/research/26/c/inside-LiteLLM-supply-chain-compromise.html
26	StepSecurity	hackerbot-claw: AI-Powered Bot Exploiting GitHub Actions	https://www.stepsecurity.io/blog/hackerbot-claw-github-actions-exploitation
27	Palo Alto Networks	Breaking Down the Trivy Supply Chain Attack	https://www.paloaltonetworks.com/blog/cloud-security/trivy-supply-chain-attack/
28	Arctic Wolf	TeamPCP Supply Chain Attack Campaign	https://arcticwolf.com/resources/blog/teampcp-supply-chain-attack-campaign-targets-trivy-checkmarx-kics-and-LiteLLM-potential-downstream-impact-to-additional-projects/
29	Kaspersky	Trojanization of Trivy, Checkmarx, and LiteLLM	https://www.kaspersky.com/blog/critical-supply-chain-attack-trivy-LiteLLM-checkmarx-teampcp/55510/
30	SANS Institute	When the Security Scanner Became the Weapon	https://www.sans.org/webcasts/when-security-scanner-became-weapon
31	Cyble	TeamPCP Threat Actor Profile	https://cyble.com/threat-actor-profiles/teampcp/
32	ARMO	The Trivy Supply Chain Attacks Explained	https://www.armosec.io/blog/trivy-supply-chain-attack-ci-cd-security-lessons/
33	Datadog IOCs (CSV)	TeamPCP Indicators of Compromise Feed	https://github.com/DataDog/indicators-of-compromise/raw/refs/heads/main/teampcp/iocs.csv

#	Source	Title	URL
34	GitHub Advisory	GHSA-9p44-j4g5-cfx5 — Trivy Action Script Injection (CVE-2026-26189)	https://github.com/advisories/GHSA-9p44-j4g5-cfx5
34	U.S. GAO	SolarWinds Cyberattack Demands Significant Response (Infographic)	https://www.gao.gov/blog/solarwinds-cyberattack-demands-significant-federal-and-private-sector-response-infographic
35	SolarWinds	An Investigative Update of the Cyberattack	https://www.solarwinds.com/blog/an-investigative-update-of-the-cyberattack
36	NCSC (UK)	Log4j Vulnerability — What Everyone Needs to Know	https://www.ncsc.gov.uk/information/log4j-vulnerability-what-everyone-needs-to-know
37	Trivy Documentation	Kubernetes Scanning — Trivy Operator	https://trivy.dev/docs/latest/target/kubernetes/
38	SANS Institute (Hartman & Johnson)	When the Security Scanner Became the Weapon: TeamPCP Threat Intelligence Report v3.0	https://www.sans.org/webcasts/when-security-scanner-became-weapon
39	CrowdStrike	From Scanner to Stealer: Inside the trivy-action Supply Chain Compromise	https://www.crowdstrike.com/en-us/blog/from-scanner-to-stealer-inside-the-trivy-action-supply-chain-compromise/
40	Team Cymru	Jingle Shells: How Virtual Offices Enable a Facade of Legitimacy (TECHOFF SRV LIMITED)	https://www.team-cymru.com/post/jingle-shells-how-virtual-offices-enable-a-facade-of-legitimacy
41	Chainguard (Dan Lorenc)	What the Fork? Imposter Commits in GitHub Actions and CI/CD	https://www.chainguard.dev/unchained/what-the-fork-imposter-commits-in-github-actions-and-ci-cd
42	Boost Security Labs	MegaGame10418: The User Behind hackerbot-claw	https://labs.boostsecurity.io/articles/megagame10418-the-user-behind-hackerbot-claw/
43	Wiz Research	Three's a Crowd: TeamPCP Trojanizes LiteLLM	https://www.wiz.io/blog/threes-a-crowd-teampcp-trojanizes-LiteLLM-in-continuation-of-campaign
44	Rami McCarthy	TeamPCP Campaign Timeline (Phases 01–09)	https://ramimac.me/trivy-teampcp/
45	CISA	Supply Chain Compromise: tj-actions/changed-files (CVE-2025-30066)	https://www.cisa.gov/news-events/alerts/2025/03/18/supply-chain-compromise-third-party-tj-actionschanged-files-cve-2025-30066-and-reviewdogaction
46	Unit 42 (Palo Alto Networks)	GitHub Actions Supply Chain Attack: Targeted Attack on Coinbase (SpotBugs origin)	https://unit42.paloaltonetworks.com/github-actions-supply-chain-attack/
47	Wiz Research	s1ngularity: Supply Chain Attack Leaks Secrets on GitHub (Nx)	https://www.wiz.io/blog/s1ngularity-supply-chain-attack
48	StepSecurity	s1ngularity: Nx Build System Package Compromised with Data-Stealing Malware	https://www.stepsecurity.io/blog/supply-chain-security-alert-popular-nx-build-system

#	Source	Title	URL
			m-package-compromised-with-data-stealing-malware
49	GitGuardian	The Nx s1ngularity Attack: Inside the Credential Leak	https://blog.gitguardian.com/the-nx-s1ngularity-attack-inside-the-credential-leak/
50	Wiz Research	Shai-Hulud 2.0: Ongoing Supply Chain Attack (25,000+ repos)	https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack
51	GitGuardian	Shai-Hulud 2.0: The Second Coming — 33,185 Secrets Exposed	https://blog.gitguardian.com/shai-hulud-2/
52	Microsoft Security Blog	Shai-Hulud 2.0: Guidance for Detecting and Defending	https://www.microsoft.com/en-us/security/blog/2025/12/09/shai-hulud-2-0-guidance-for-detecting-investigating-and-defending-against-the-supply-chain-attack/
53	Unit 42 (Palo Alto Networks)	Shai-Hulud Worm Compromises npm Ecosystem	https://unit42.paloaltonetworks.com/npm-supply-chain-attack/
54	GitHub Blog	Strengthening Supply Chain Security: Preparing for the Next Malware Campaign	https://github.blog/security/supply-chain-security/strengthening-supply-chain-security-preparing-for-the-next-malware-campaign/
55	Google / The Hacker News	UNC6426 Exploits Nx s1ngularity to Gain AWS Admin Access in 72 Hours	https://thehackernews.com/2026/03/unc6426-exploits-nx-npm-supply-chain.html
56	Dark Reading	Supply Chain Worms 2026: What Shai-Hulud Taught Attackers and How to Prepare	https://www.darkreading.com/cyberattacks-data-breaches/supply-chain-worms-in-2026-what-shai-hulud-taught-attackers-and-how-to-prepare
57	Aikido Security	Shai-Hulud: GitHub Actions Initial Infection Vector	https://www.aikido.dev/blog/github-actions-incident-shai-hulud-supply-chain-attack